VIETNAM NATIONAL UNIVERSITY, HANOI
UNIVERSITY OF ENGINEERING AND TECHNOLOGY



NGUYỄN MẠNH HÙNG

AGENTIC AI FOR PROGRAMMING EDUCATION

BACHELOR THESIS

Major: Artificial Intelligence

HA NOI - 2025

VIETNAM NATIONAL UNIVERSITY, HANOI
UNIVERSITY OF ENGINEERING AND TECHNOLOGY

Nguyễn Mạnh Hùng

AGENTIC AI FOR PROGRAMMING EDUCATION

BACHELOR THESIS

Major: Artificial Intelligence

Supervisor: Ph.D Hoang Thanh Tung

HA NOI - 2025

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ

Nguyễn Mạnh Hùng

TÁC TỬ AI TRONG GIÁO DỤC LẬP TRÌNH

KHÓA LUẬN TỐT NGHIỆP ĐẠI HỌC HỆ CHÍNH QUY
Ngành: Trí Tuệ Nhân Tạo

Cán bộ hướng dẫn: TS. Hoàng Thanh Tùng

HÀ NỘI - 2025

Thesis advisor: Ph.D. Hoàng Thanh Tùng Nguyễn Mạnh Hùng

Abstract

Abstract: In the context of modern learning systems that increasingly require personalization, reliability, and scalability, using a single LLM-based agent to handle the entire workflow is no longer sufficient. When a single agent is responsible for performing all tasks simultaneously, its performance is inherently limited, as it cannot be optimized for each specific role. An interactive learning system is a complex process that involves several different tasks, such as document analysis, question generation, quality evaluation, and learner support. Each task requires its own criteria, reasoning process, and quality standards. For this reason, adopting a Multi-Agent System (MAS) becomes a promising approach. MAS allows the system to separate tasks into specialized agents, where each agent is optimized for a specific function and can work together smoothly with other agents. This design not only improves the accuracy of the system but also increases its controllability, interpretability, and scalability. A modular, multi-agent architecture also provides flexibility, allowing components to be upgraded, replaced, or reused without affecting the entire system. This thesis introduces MLA, an interactive learning framework composed of several modules, each fulfilling a distinct function in the learning process. The first module is the question generation system, which is operated by three agents: a Question Generator that produces questions based on learning objectives and input materials; an Answer Generator that creates multiple-choice options including correct answers and suitable distractors; and a Question Reviewer that evaluates all generated questions based on six criteria: content coverage, clarity, accuracy and relevance, difficulty, cognitive level, and learner engagement. This evaluation step ensures that the final questions meet pedagogical standards and align with the intended learning outcomes. The second module supports learning material analysis. It generates summaries, mind maps, and applies contextual retrieval methods for question-answering tasks, helping learners understand the structure of the content, identify key ideas, relationships between concepts, and difficult parts in the material. The results show clear

improvements when the system ensures high-quality distractors, reduces model errors, and handles long context better. The quality of the generated questions, the content coverage, and the cognitive level required are all higher than systems that use only one agent. In addition, the mind maps, summaries, and answering ability of the system are more stable and give better support to learners. These results show that the multi-agent design creates a more interactive, flexible, and effective learning experience.

Keywords: *Multi-agent system, Interactive learning, Multiple-choice question generation, Context retrieval.*

Tóm tắt

Trong bối cảnh các hệ thống học tập hiện đại ngày càng đòi hỏi tính cá nhân hóa, độ tin cậy và khả năng mở rộng, việc sử dụng đơn tác tử dựa trên mô hình ngôn ngữ lớn (LLM) cho toàn bộ quy trình không còn đáp ứng hiệu quả, tác tử phải xử lý đồng thời tất cả các nhiệm vụ dẫn đến kết quả không tốt vì không tối ưu cho từng vai trò riêng biệt. Một hệ thống học tập tương tác thực sự là một quy trình phức tạp, bao gồm nhiều nhiệm vụ khác nhau như phân tích tài liệu, sinh câu hỏi, đánh giá chất lượng và hỗ trợ người học. Mỗi nhiệm vụ đòi hỏi những tiêu chí, cách suy luận và chuẩn chất lượng riêng. Chính vì vậy, việc áp dụng hệ thống đa tác tử (MAS) trở thành một hướng tiếp cận nổi bật, cho phép phân tách từng nhiệm vụ thành các tác tử chuyên biệt, mỗi tác tử tối ưu cho một vai trò cụ thể và có thể phối hợp nhịp nhàng với các tác tử khác. Nhờ đó, hệ thống không chỉ đạt được độ chính xác, mà còn cải thiện khả năng kiểm soát, giải thích và mở rộng trong tương lai. Việc tổ chức theo hướng này tạo ra một kiến trúc linh hoạt, trong đó các thành phần có thể nâng cấp, thay thế hoặc tái sử dụng mà không ảnh hưởng đến toàn bộ hệ thống. Luận văn này đề xuất một khung hệ thống học tập tương tác MLA gồm nhiều mô-đun, mỗi mô-đun đóng vai trò riêng nhằm hỗ trợ quá trình học tập. Mô-đun thứ nhất là hệ thống sinh câu hỏi, được ba tác tử phối hợp: Tác tử tạo câu hỏi dựa trên mục tiêu học tập và nội dung tài liệu; Tác tử tạo các đáp án xây dựng các phương án trắc nghiệm gồm đáp án đúng và các phương án nhiễu phù hợp; và Tác tử đánh giá lại toàn bộ câu hỏi theo sáu tiêu chí: mức độ bao phủ nội dung, độ rõ ràng, tính chính xác và phù hợp, mức độ khó, mức độ yêu cầu tư duy và mức độ thu hút người học. Quy trình đánh giá này đảm bảo rằng câu hỏi đầu ra vừa đáp ứng chuẩn sư phạm vừa phù hợp với mục tiêu học tập đặt ra. Mô-đun thứ hai hỗ trợ phân tích tài liệu học tập, tạo các bản tóm tắt, sơ đồ tư duy và áp dụng cơ chế truy xuất ngữ cảnh cho tác vụ hỏi–đáp nhằm giúp người học hiểu rõ cấu trúc nội dung, nhận diện các ý chính, mối quan hệ giữa các khái niệm quan trọng cũng như những phần khó trong tài liệu. Kết quả cho thấy nhiều cải thiện rõ rệt khi hệ thống đảm bảo chất lượng của các phương án nhiễu, giảm sai lệch

của mô hình và tăng cường khả năng xử lý ngữ cảnh dài. Chất lượng câu hỏi được tạo ra, mức độ bao phủ nội dung và yêu cầu về nhận thức đều vượt trội so với các hệ thống chỉ sử dụng một tác tử. Bên cạnh đó, các sơ đồ tư duy, bản tóm tắt và khả năng giải đáp của hệ thống đều thể hiện độ ổn định cao hơn và mang lại giá trị hỗ trợ người học hiệu quả hơn. Những kết quả này cho thấy thiết kế đa tác tử mang lại trải nghiệm học tập tương tác, linh hoạt và hiệu quả hơn.

Từ khóa: *Hệ thống đa tác tử, Học tập tương tác, Sinh câu hỏi trắc nghiệm, Truy xuất ngữ cảnh.*

Acknowledgements

First, I would like to thank Dr. Hoang Thanh Tung for giving me guidance, knowledge, and support throughout my studies in Artificial Intelligence and Computer Science. His help was very important for the completion of this thesis and for my success at the University of Engineering and Technology.

I also want to thank my friends in class K67-AI. During four years of studying together, they supported me both in learning and in daily life. I am grateful for all the experiences we shared and the things we achieved together. My thanks also go to my friends at IAI, UET, who encouraged me and helped me academically and emotionally during my university years.

My time at the University of Engineering and Technology has been meaningful and memorable. I am thankful for the good learning environment, modern facilities, and opportunities that allowed me to grow and develop myself. This university has been an important part of my life, and I truly appreciate everything it has given me.

Finally, I want to give my deepest thanks to my family. They have always supported me, encouraged me, and believed in me throughout my studies and during the process of writing this thesis. Their support is something I value greatly and could not put into words.

Declaration

I declare that the thesis has been composed by myself and that the work has not been submitted for any other degree or professional qualification. I confirm that the work submitted is based on my own efforts with the dedicated help and guidance for the only main purpose of graduation.

All information, data, and results presented in the thesis are completely honest, and not copied from the results of existing topics that are not clearly stated in the references. All references related to the graduate thesis are fully listed in the reference list.

I take full responsibility for my commitments in accordance with the regulations of the VNU University of Engineering and Technology.

Student

Nguyen Manh Hung

Table of Contents

Abstract . . . . . **iii**

Tóm tắt . . . . . **v**

Acknowledgements . . . . . **vii**

Declaration . . . . . **viii**

Acronyms & Abbreviation . . . . . **xii**

List of Figures . . . . . **xiv**

List of Tables . . . . . **xvi**

1 INTRODUCTION **1**

1.1 Background and Research Motivation 1

1.2 Research Problem 2

1.3 Research Objectives and Scope 2

1.4 Thesis structure 4

2 LITERATURE REVIEW **5**

2.1 Background 5

2.1.1 Large Language Model 5

2.1.2 Vision Language Model 6

2.1.3 Retrieval Augmented Generation 7

2.1.4 Multi Agent Systems 7

2.2 Related work 8

2.2.1 Generative AI in Education 8

2.2.2 Document Understanding and Text Extraction 10

2.2.3 Document Question Answering 11

2.2.4 Document Question Generation 12

3 Materials and Methods **14**
3.1 Design goals 14
3.2 System Analysis and Design 18
3.2.1 System Analysis 18
3.2.2 System Design 21
3.3 Event-Driven Routing Mechanism 21
3.4 Document Preprocessing and Structuring Module 22
3.5 Multiple Choice Question Generation Module 24
3.6 Mindmap Module 25
3.6.1 Document Summarization Module 25
3.6.2 Mindmap Generation Module 27
3.7 Document Question Answering Module 27
3.7.1 Pre-QA Processing Module 27
3.7.2 Hybrid Retrieval-Augmented Generation 29

4 Experiments and Results **31**
4.1 Evaluation Methods 31
4.1.1 Modular Evaluation Framework 31
4.1.2 Human Evaluation 33
4.1.3 LLM-as-a-Judge 33
4.2 Collection 34
4.3 Results and Analysis 36
4.3.1 Evaluation of Question Generation Quality 36
4.3.2 Summary of Results 40
4.4 Discussions 40
4.4.1 LLMs Performance in Long Context Retrieval 40
4.4.2 Biases in LLM-Generated Multiple-Choice Questions 41
4.4.3 Limitations in Distractor Generation Pipeline 43
4.4.4 Interpretation of ARC Metrics for Summary Evaluation 43
4.4.5 Human vs LLM Evaluation Divergence 44

Conclusions **45**

A Activity Diagram **51**

B Usecase Diagram **54**

C Sequence Diagram **56**

D State Machine Diagrams **59**

E Database Design **63**

F User Interface Design **64**

G Test Prompting **67**

Acronyms & Abbreviation

AGI	Artificial General Intelligence
AI	Artificial Intelligence
ARC	Argument Representation and Coverage
BM25	Best Matching 25 Ranking Function
ChartQA	Chart-based Question Answering Benchmark
CNNs	Convolutional Neural Networks
DAI	Distributed Artificial Intelligence
DiT	Document Image Transformer
DocVQA	Document Visual Question Answering
DPR	Dense Passage Retrieval
DQA	Document-Based Question Answering
DRC	Document Retrieval and Comprehension
ELECTRA	Efficiently Learning an Encoder that Classifies Token Replacements Accurately
FT	Fine-Tuning
HotpotQA	Hotpot Multi-hop Question Answering Dataset
ITS	Intelligent Tutoring System
LLM	Large Language Model

LVLMs	Large Vision-Language Models
MARL	Multi-Agent Reinforcement Learning
MAS	Multi-Agent System
MCQ	Multiple-Choice Question
MLA	MLearn Assistant
MMC	Mind Map Construction
MMG	Mind Map Generation
MMMU	Massive Multi-discipline Multimodal Understanding Benchmark
MS MARCO	Microsoft Machine Reading Comprehension Dataset
NLP	Natural Language Processing
OCR	Optical Character Recognition
PEFT	Parameter-Efficient Fine-Tuning
QA	Question Answering
QBAs	Question-Based Activities
RAG	Retrieval-Augmented Generation
Seq2Seq	Sequence-to-Sequence Model
SQuAD	Stanford Question Answering Dataset
STEM	Science, Technology, Engineering, and Mathematics
UnifiedQA	Unified Question Answering Model
VideoMME	Video Multimodal Evaluation Benchmark
VisualMRC	Visual Machine Reading Comprehension
VLM	Vision-Language Model

List of Figures

3.1 Illustration of the document mindmap process 15
3.2 Illustration of the question generation process. 16
3.3 Illustration of the document question–answering process 17
3.4 Functional hierarchy diagram of the interactive learning system 21
3.5 Prompt Routing Mechanism for Multi-Module Learning System 22
3.6 Document Processing Pipeline 23
3.7 Multi-Agent Interactive Chain: Q-Gen (question generation), A-Gen (answer generation), Judge (quality evaluation), iterative refinement, and final normalization. 24
3.8 Hybrid summarization workflow combining two parallel processing branches: Abstractive Summarization and Extractive Summarization. The outputs from both models are merged through a Summary Fusion step to produce a concise, accurate, and semantically rich final summary. 26
3.9 Data Preprocessing and Structuring Pipeline: Page Splitting, OCR and Layout Reconstruction, generation of Markdown Sections, context-aware Chunking, and embedding encoding for storage in the Vector Database. . 28
3.10 The Hybrid RAG workflow for retrieval and information synthesis. 29

4.1 Comparison of question quality ratings across evaluation criteria. Bars corresponding to the MLA system use a horizontal hatch pattern ("–"), while bars representing the GPT-4o baseline use a diagonal hatch pattern ("//"), enabling clear visual distinction between the two systems. 37
4.2 Human Evaluation of the MLA System. The rating scale is color-coded from lowest to highest as follows: red (low), orange (below expectation), green (good), and purple (excellent). 39
4.3 LLMs processing long context 42

A.1 Activity diagram of the process 52
A.2 Continuation of the activity diagram 53

A.3 Final part of the activity diagram 53

B.1 Overview use case diagram of the system. 55

C.1 Initial part of the sequence diagram 57
C.2 Final part of the sequence diagram 58

D.1 Learner Behavior State Machine 61
D.2 Learning Session Lifecycle State Machine 62

E.1 Database 63

F.1 Illustration of the document mindmap process 64
F.2 Illustration of the document mindmap process 65
F.3 Illustration of the document mindmap process 65
F.4 Illustration of the document mindmap process 66

G.1 Fullset Evaluation Prompt and Argument Role Evaluation Prompt 67
G.2 Direct Question Generation Prompt and the Evaluation Criteria for Understanding and Clarity 68
G.3 Criteria for Evaluating Multiple-Choice Questions 69

List of Tables

4.1 Statistics of the number of pages and words of the 10 textbooks 36
4.2 ARC metrics for summary evaluation of the MLA system across datasets 41

Chapter 1

INTRODUCTION

1.1 Background and Research Motivation

In programming education and other STEM fields, understanding learning materials plays an important role in effective learning. Many learning documents today are long and have complex structures. They often include text, tables, images, and source code. Especially in online learning environments, students usually study by themselves with limited guidance.

LLMs and VLMs have shown good performance when working with short texts or simple tasks. However, their performance decreases clearly when they need to process long and complex content. This leads to poor quality in tasks such as question generation and summarization. The problem becomes more serious when documents include multiple formats, such as source code and diagrams. In addition, there are other challenges, including the lack of high-quality labeled data, differences between knowledge domains, and limited support for low-resource languages. These factors make it difficult to apply LLMs and VLMs effectively in real educational settings.

Learning is not only about reading information. Students also need ways to check how well they understand the content. They need feedback to know whether they understand concepts correctly and can apply their knowledge in practice. Therefore, there is a clear need for learning systems that can transform static documents into interactive learning experiences and provide timely and meaningful feedback. This need is the main motivation of this research.

1.2 Research Problem

Most learning support systems that use LLMs today rely on a single model or a single agent to manage the whole learning process. This process usually includes understanding learning documents, creating questions, and checking answers. While this method can support simple tasks, it does not work well for long and complex documents in programming education.

Single-agent systems often struggle to keep a stable understanding of long documents. The quality of generated questions can change from one time to another, and it is difficult to control how hard the questions are. In many cases, the generated answers and distractors are not strong enough to test students' understanding, especially in programming tasks. These systems also have problems when checking answers that include source code, where both logic and correctness are important. In addition, all tasks are handled by the same agent, so there is no clear separation between analysis, question creation, and evaluation. These systems usually do not include self-checking or comparison steps, which makes the results less reliable and difficult to repeat in real learning settings.

Another clear weakness is the low level of interaction. Many systems only give one-way feedback and do not support active question–answer interaction between the learner and the system. Because of this, learners have fewer chances to practice, reflect on their mistakes, and improve step by step.

For these reasons, this thesis studies how to design an interactive learning system based on a multi-agent architecture. The system is expected to support understanding of long documents, generate structured questions with suitable difficulty, and evaluate answers in a reliable way. The main challenge is to coordinate multiple agents with different roles while keeping shared context, stable behavior, and clear learning goals. This research is especially important in programming education, where both conceptual understanding and technical accuracy must be assessed.

1.3 Research Objectives and Scope

The main goal of this thesis is to build a MAS framework based on LLM and VLM. The system aims to improve the ability to understand and evaluate long, complex, and multi modal programming documents. The specific goals of the research include: analyzing the characteristics of long programming documents and the requirements for content

understanding in STEM education; designing a multi agent architecture with clear roles such as document understanding, question generation, document question answering, verification, quality checking, and mind map generation; developing a structured question generation model that can control difficulty levels and match the document content; building a document question answering mechanism based on multi level retrieval to ensure accuracy and alignment with the original text; creating a mind map generation process to help learners identify key ideas and concept structures; designing an answer evaluation mechanism that combines rubric based text grading with automatic test case grading for source code; and finally, evaluating the effectiveness, stability, and reproducibility of the whole system using both qualitative and quantitative criteria. Through this work, the thesis aims to create a reliable, scalable method for automatic learning and understanding in programming education.

The scope of the research is defined to keep the thesis focused and verifiable. In terms of content, the study works with programming documents such as textbooks, lectures, and technical modules that include text, tables, images, and source code (mainly Java). The system is designed to answer questions based on the document, generate new questions, and extract key ideas for mind map construction all directly related to technical content, concepts, and code examples. Technically, the system is built using existing LLM and VLM models, organized in a sequential multi agent architecture with feedback and shared context across steps. For evaluation, the thesis focuses on measuring question quality, reliability in document question answering, and mind map quality using indicators such as coherence, content coverage, difficulty control, accuracy, and usability. The study does not extend to full personalized teaching or emotional and behavioral aspects of learners.

With this direction, the thesis aims to contribute a practical and verifiable approach to understanding documents, especially long documents, in programming education. More specifically, the research provides three main contributions: (1) a general model for understanding and evaluating documents through question generation, answer evaluation, document question answering, and mind map creation; (2) a multi agent architecture that is scalable and reusable for intelligent learning systems; and (3) a set of evaluation criteria for question generation, answer grading, document question answering, and mind map quality, helping to standardize assessment processes in programming learning environments.

1.4 Thesis structure

This thesis is structured into four main chapters, followed by appendices. Chapter 1 introduces the research motivation, problem statement, and objectives of applying Large Language Models (LLMs) to educational support tasks. Chapter 2 reviews the background and related work, including generative AI in education, document understanding and extraction, document question answering and question generation, retrieval methods, multi-agent workflows, summarization, mind map generation, and evaluation approaches. Chapter 3 presents the proposed methodology and system design, describing the end-to-end framework and how the core modules question generation, document question answering, summarization, and mind map creation are integrated through a modular, multi-agent workflow, together with the data processing and retrieval pipeline. Chapter 4 reports the experimental setup and results, including dataset preparation, prompt and agent workflow configuration, retrieval and long-context analysis, ARC-based summary evaluation, and both LLM-based and human assessments of question quality, followed by a consolidated discussion of limitations, conclusions, and future work. The appendices provide supplementary materials such as activity diagrams, use-case diagrams, and state-machine diagrams to clarify system workflows and internal processes.

Chapter 2

LITERATURE REVIEW

2.1 Background

2.1.1 Large Language Model

A LLM is a language model trained with deep learning on a very large text dataset, which can include billions or even trillions of words from books, articles, websites, source code, and many other sources. The goal of an LLM is to help computers understand, process, and generate natural language in a way that is close to how humans use language. Because of this, the model can perform many tasks such as answering questions, summarizing text, translating, creating content, or supporting reasoning in complex situations.

At the core of an LLM is the Transformer architecture, introduced by Vaswani et al (2017) [1]. The key idea is the self attention mechanism, which allows the model to find and focus on important parts of a text sequence. Each word is turned into a high dimensional vector, which helps the model understand the semantic relationships between words. Through hundreds of neural layers, these vectors are gradually transformed to form deeper contextual representations, allowing the model to learn complex language patterns.

During training, the model receives a massive amount of text data and learns to predict the next word in a sentence. The prediction error is measured by a loss function and optimized using backpropagation and gradient descent. This process adjusts billions of parameters so the model can learn language structure, meaning, and writing style. Famous models such as GPT-3[2], PaLM[3], LLaMA[4] have shown the strong

capabilities of LLMs in many fields. These fields include chatbots, virtual assistants, machine translation, content creation, medical analysis, programming, education, and finance. Thanks to their ability to model language at a very large scale, LLMs have become an important foundation of modern artificial intelligence and have opened new ways for natural and effective communication between humans and machines.

2.1.2 Vision Language Model

VLM is a multimodal generative AI model that can understand, analyze, and generate responses from image, video, and text data [5]. It is an extension of LLMs, allowing the model not only to understand language but also to see and interpret visual information.VLM typically includes three main components: a vision encoder that extracts features from images or videos; a projector that converts these visual representations into a form that the LLM can process; and a large language model that jointly handles text and visual inputs to produce appropriate responses. With this structure, VLMs can perform many tasks such as visual question answering, video summarization, image analysis, information extraction from documents, and multi modal conversation [5].

Unlike traditional vision models based on CNNs, which are usually limited to a single task, VLMs can generalize more flexibly and accomplish multiple tasks using natural language prompts. These models are trained on large datasets of image–caption pairs, enabling them to learn semantic relationships between vision and language and achieve strong zero shot performance across many tasks without retraining.The training process of a VLM often includes three stages: pre training to align model components, supervised fine tuning to ensure contextually correct responses, and PEFT to adapt the model to specialized data domains. The effectiveness of VLMs is evaluated using benchmarks such as MMMU, DocVQA, ChartQA, and Video MME, which measure their ability to understand, reason, and interpret visual information. [5].

Thanks to their ability to combine language and vision, VLMs are becoming an important foundation for modern AI systems, with applications in education, document analysis, industrial monitoring, transportation, security, and healthcare. They are considered a significant step AGI, where machines can see, read, understand, and respond in a unified way similar to humans [5].

2.1.3 Retrieval Augmented Generation

LLMs despite their impressive capabilities, still face several inherent challenges, such as hallucinations generating incorrect information, outdated knowledge, and reasoning processes that are often opaque and difficult to trace. To address these limitations especially in knowledge intensive tasks RAG has emerged as a key solution [6]. RAG is a hybrid framework that combines the internal, parameterized knowledge of an LLM with access to large and dynamic external knowledge sources (non parameterized). By integrating information retrieved from external documents, RAG significantly improves the accuracy and reliability of generated content, while also allowing the system to remain up to date and incorporate domain specific knowledge without the expensive need to retrain the entire model.

The basic RAG pipeline, often called Naive RAG, follows a three step Retrieve–Read workflow: Indexing, Retrieval, and Generation [6]. In the Indexing stage, documents are processed, chunked, encoded into vectors, and stored in a vector database. Upon receiving a user query, the Retrieval stage encodes the query and searches for the most relevant chunks. Finally, in the Generation stage, the LLM receives the original question together with the retrieved chunks as context and produces a complete and accurate answer.However, Naive RAG still faces challenges in retrieval quality and generation performance, such as retrieving irrelevant chunks or missing important information. To address these issues, more advanced variants have been developed. Advanced RAG introduces pre and post - retrieval optimization strategies (e.g., reranking), while Modular RAG provides a more flexible architecture that allows components to be replaced or rearranged to target specific problems[6]. In the LLM optimization, RAG differents from FT: RAG is often compared to giving a model a textbook it can consult in real time, whereas FT is like a student absorbing knowledge internally through training.These two approaches are not mutually exclusive they can be combined to achieve optimal performance.

2.1.4 Multi Agent Systems

MAS is a branch of DAI in which multiple agents operate independently while being able to interact, coordinate, and collaborate to achieve one or more shared goals within a given environment. Each agent is considered an intelligent entity capable of perception, reasoning, decision making, and autonomous action. The development of MAS originates from the need to model complex real world systems where no single entity can

manage or control the entire process.MAS has been applied in many domains, including cooperative robotics, intelligent traffic management, e-commerce, finance, sensor networks, and more recently, intelligent tutoring systems. In modern settings, MAS is increasingly combined with LLMs, which enable agents to reason, collaborate, and communicate through natural language [7].A typical MAS consists of four main components:

- **Agent:** the basic unit of the system, capable of autonomous perception, reasoning, and action.
- **Environment:** the space in which agents operate, exchange information, and influence one another.
- **Interaction:** including communication, coordination, and cooperation among agents.
- **Organization:** the structure that defines roles, responsibilities, and collaboration mechanisms within the system.

Unlike centralized systems, MAS is distributed, flexible, and highly scalable, making it suitable for large scale or dynamic problems where information may be incomplete or constantly changing. By decomposing tasks into smaller components, MAS enables parallel problem solving and improves overall efficiency.

In contemporary research, MAS is not only a technical architecture but also a conceptual framework for collective intelligence, in which multiple intelligent entities learn, adapt, and optimize group behavior. Emerging directions in MAS include MARL, multi agent planning, and especially LLM based multi agent systems, which provide high level semantic collaboration and strong adaptability to complex situations [7].

2.2 Related work

2.2.1 Generative AI in Education

2.2.1.1 Before the Era of Large Language Models

Before LLMs became widely adopted around 2018–2019, Artificial Intelligence had already contributed significantly to education through technologies such as Intelligent Tutoring Systems, Machine Learning, Natural Language Processing, and intent based classification systems.

ITS first appeared in the 1970s and later evolved into advanced, knowledge centered platforms during the 1990s, enabling personalized instruction by adapting to the individual needs of each learner [8].

Similarly, machine learning techniques were applied to educational data to identify at risk students early, personalize learning content, and even automate routine tasks such as essay scoring through automated writing assessment tools. These algorithmic advances not only helped detect learning difficulties earlier but also supported automatic personalized intervention, laying a strong foundation for adaptive and effective education.

At the same time, NLP based applications enhanced natural language interaction in learning environments. Automated writing evaluation tools provided scalable feedback on student essays, while conversational agents used intent classification to understand student questions and deliver appropriate responses, thereby improving interaction and offering real time support.

2.2.1.2 During the Era of Large Language Models

The rise of LLMs has marked a major turning point in technology enhanced learning, especially since the release of ChatGPT in 2022. This shift in core architecture has moved educational AI from traditional NLP techniques to modern Transformer based models capable of performing tasks that were previously impossible. With LLMs becoming increasingly accessible, both teachers and learners can now integrate them into a wide range of learning activities [9].

A key advantage of LLMs is their scalability, which enables highly personalized learning experiences for large and diverse groups of students. By automating labor intensive tasks such as generating learning materials, providing real time feedback, and even scoring assignments, LLMs make personalized learning feasible at scale [9].

For learning systems like ours, LLMs introduce new forms of interaction through advanced reasoning and contextual understanding. LLMs can support document based question answering, generate structured multiple choice questions aligned with the content of long documents, and assist learners by explaining concepts in adaptive ways. They also enable tasks such as summarization, mind map generation, and retrieval-augmented reasoning, which strengthen a learner's ability to explore and understand complex multimodal materials.

Furthermore, LLMs provide engaging learning experiences through role playing and scenario simulation. By taking on different roles such as a tutor, a peer partner, or a domain expert LLMs promote interactive, immersive learning sessions that connect theoretical knowledge with practical application.

Recent studies have also explored the use of LLMs to simulate student behavior, such as analyzing whether their error patterns on multiple choice questions resemble those of real learners. Such work including approaches that simulate students with diverse cognitive levels using LLM based agents, shows how LLMs can help generate higher quality assessment items and support more effective learning interventions [10].

2.2.2 Document Understanding and Text Extraction

Research on document understanding and text extraction has progressed through several developmental phases, from traditional OCR and rulebased information extraction methods to modern multimodal deep-learning models.

In the initial phase, systems primarily followed pipelines that consisted of OCR then layout analysis, finally rulebased extraction,notable examples being tools such as Tesseract [11]. Although this approach was effective for well structured printed documents,it encountered limitations when processing complex forms or unstructured documents.In the subsequent phase, with the advent of deep learning, models such as TRIE (Zhang et al., 2020) introduced a unified end to end framework that integrates text reading and information extraction in a mutually reinforcing way [12]. These models leveraged both visual and textual features to improve extraction accuracy across diverse document types.Later, layout-aware pre-trained models such as LayoutLM (Xu et al., 2019) [13] and BROS (Hong et al., 2021) [14] opened a new front by simultaneously exploiting semantic (text) and layout (structure) information in documents, achieving superior performance on benchmarks such as FUNSD [15], CORD [16], and RVL-CDIP.More recently, the trend in document understanding has shifted toward combining LLMs and VLMs to process multimodal documents that do not rely on a traditional OCR pipeline. For instance, DocVLM (Nacson et al., 2024) demonstrates high potential for efficiently handling complex documents using compact query encodings derived from OCR inputs [17]. However, these approaches still face challenges related to long context understanding, layout concept learning, and high training costs.

In summary, current research emphasizes integrating multi modal information (text layout and vision) and learning global context, yet systems optimized specifically for

educational documents or technical lecture materials remain scarce this gap is the target of the present thesis.

2.2.3 Document Question Answering

DQA is an important research direction in NLP It aims to enable machine-learning models to understand and answer questions based on entire documents with complex structures, including text, tables, images, and page layouts. Unlike traditional QA systems that operate purely on plain text, DQA focuses on extracting knowledge from multimodal documents, supporting tasks such as retrieval, analysis, and information synthesis in richly formatted data environments.

Early research on DBQA treated the task as a form of machine reading comprehension, where a system must read, understand, and directly extract answers from the input document [18, 19]

Deep learning models such as CNNs, RNNs, attention-based networks, and GANs have been applied and shown stronger performance than traditional machine-learning approaches. Additional works, such as Choi et al. (2017) [20], leveraged salient sentences in the document as a form of summarization to support answer selection. Notably, Sun et al. (2019) [21] propose three human-inspired reading strategies back-and-forth reading, highlighting important information, and self-assessment through generating practice questions to improve how models extract and utilize contextual information in machine reading comprehension.

With the rise of LLMs and VLMs, research such as DocVQA [22] and VisualMRC [23] has expanded the scope of DQA from plain text to visually rich documents. These works enabled systems to jointly leverage textual content, visual elements, and layout information, laying the foundation for more comprehensive document understanding that integrates computer vision and natural language processing.Building on this direction, the DRC framework was proposed as a three-stage pipeline for DQA on PDF documents. The goal of DRC is to automate the understanding and extraction of knowledge from documents with complex layouts. In the Detect stage, semantic regions (paragraphs, tables, figures) are identified using the DiT combined with pdfminer.six. The Retrieve stage ranks and selects the most relevant evidence segments using methods such as BM25, Dense DPR, and ELECTRA cross-encoder. Finally, in the Comprehend stage, the UNIFIEDQA model interprets the retrieved context and generates answers in extractive, abstractive, or Boolean formats [24].The DRC approach represents a com-

prehensive research direction that unifies layout analysis, semantic retrieval, and natural language generation. As a result, such systems not only enhance the ability to understand document content but also establish a solid foundation for intelligent question-answering systems in the fields of document understanding and automated scientific knowledge extraction.

2.2.4 Document Question Generation

DQG is an extended research area of QG in NLP, focusing on automatically generating questions from long, structured documents such as textbooks, reports, research papers, and technical manuals. Unlike sentence-level QG, DQG aims to understand and exploit information that spans multiple paragraphs, pages, or even different knowledge sources, including text, tables, images, and document layout. The goal of DQG is to produce contextually relevant questions that align with the content and learning objectives, thereby supporting applications such as intelligent tutoring systems, automated assessment, document summarization, and information verification.

DQG approaches generally fall into three major categories: Structured QG, which generates questions based on knowledge bases or knowledge graphs using entities and relations to produce logically grounded questions; Unstructured QG, which generates questions from natural text using benchmark datasets such as SQuAD, MS MARCO, or HotpotQA; and Hybrid or Multimodal QG, which extends question generation to images and multimodal documents by leveraging textual content, layout information, and visual context[25].

Traditional DQG methods were primarily built on Seq2Seq architectures; however, these models were limited by the scale of training data and their generalization capability. The introduction of large pre-trained language models such as T5 [26] marked a significant breakthrough, enabling higher-quality semantic question generation through deep contextual understanding and stronger language modeling ability. More recently, LLMs and VLMs have expanded the capabilities of DQG into the multimodal space. These models can interpret relationships among text, images, tables, and document layouts, significantly improving question generation for complex educational materials.

In the field of education, DQG has been widely applied to automatically generate QBAs for both formative and summative assessment. AI-driven MCQ generation reduces teachers' workload, enhances personalization, and improves instructional efficiency. The system proposed in [27] adopts a modular approach consisting of three core

components: question generation, correct-answer prediction, and distractor generation, using Transformer models such as T5 and GPT-3 to produce questions with appropriate grammar, content alignment, and distractor quality. Survey results from more than 100 teachers indicate that over 70% are willing to adopt AI-powered QG tools to save time and increase interactivity in the learning process[27].

Current trends in DQG focus on context-aware and multi-hop question generation, leveraging the strength of LLMs through prompt engineering and in-context learning, while integrating multimodal data to deepen the model's understanding of document content. As a result, DQG is becoming a key component in intelligent learning systems, contributing to automated assessment pipelines and delivering personalized learning

Chapter 3

Materials and Methods

3.1 Design goals

The system is designed with three central goals that collectively define its intended functionality and pedagogical value. First, it aims to provide automated question generation directly from the textbook content, allowing key ideas to be identified and transformed into multiple-choice questions that support assessment and reinforce understanding. This goal reduces the manual effort required from instructors while ensuring that the generated questions remain aligned with the source material. Second, the system supports region-based question answering, enabling users to interact with a specific segment of the document rather than processing the entire text. By allowing learners and instructors to focus on a chosen region, the system facilitates targeted retrieval and clarification of information. Third, the system incorporates mind map generation to help visualize the conceptual structure of the selected content. By converting linear text into a structured representation of ideas and their relationships, this goal enhances comprehension and assists learners in forming coherent mental models of the material.

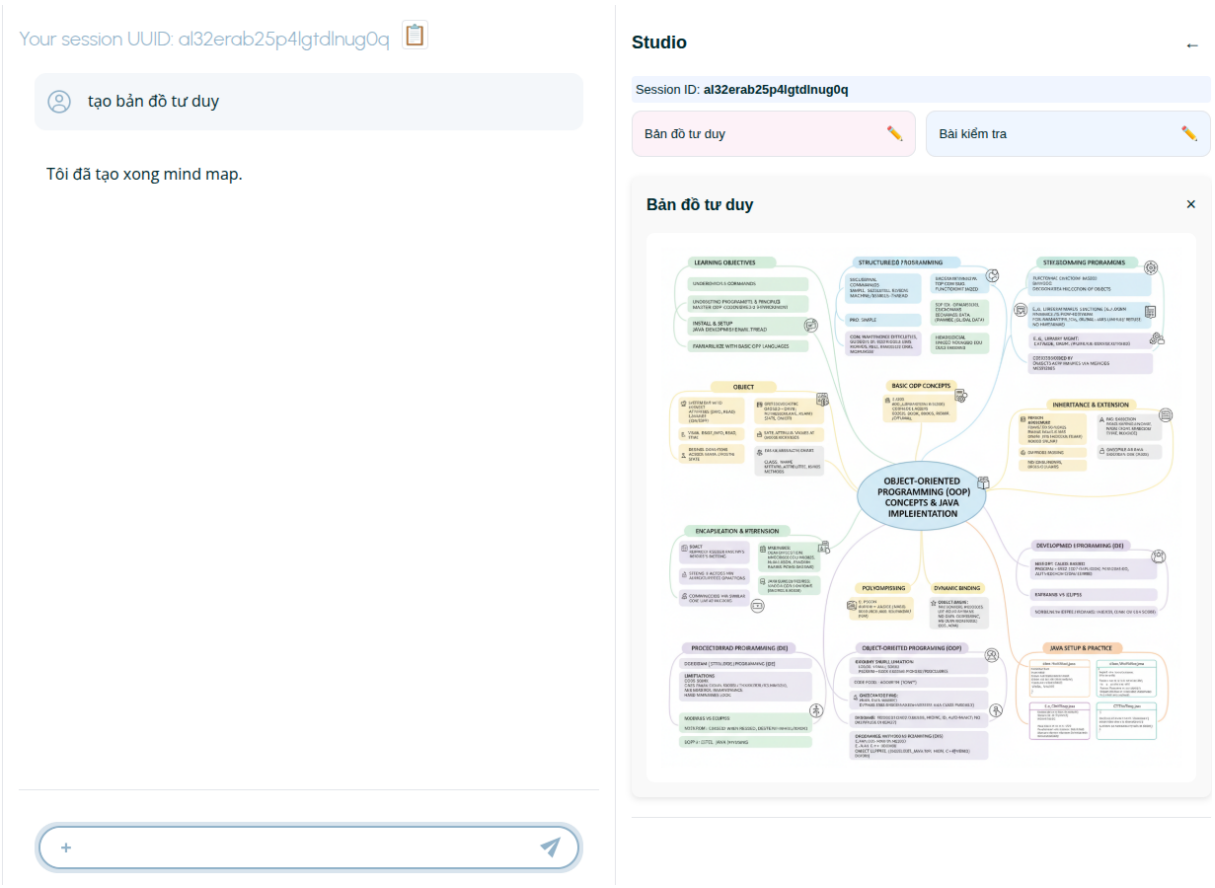


Figure 3.1: Illustration of the document mindmap process

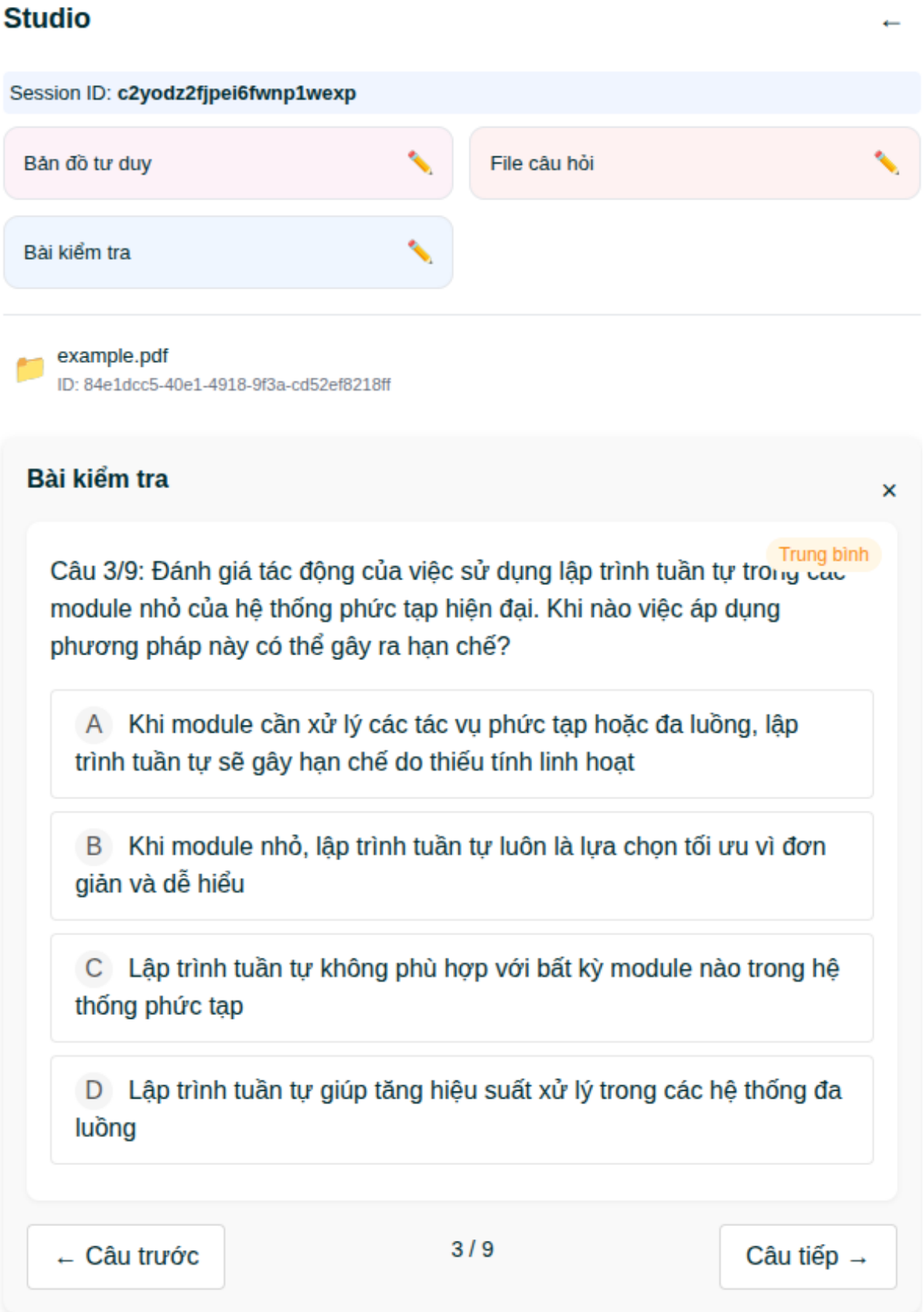


Figure 3.2: Illustration of the question generation process.

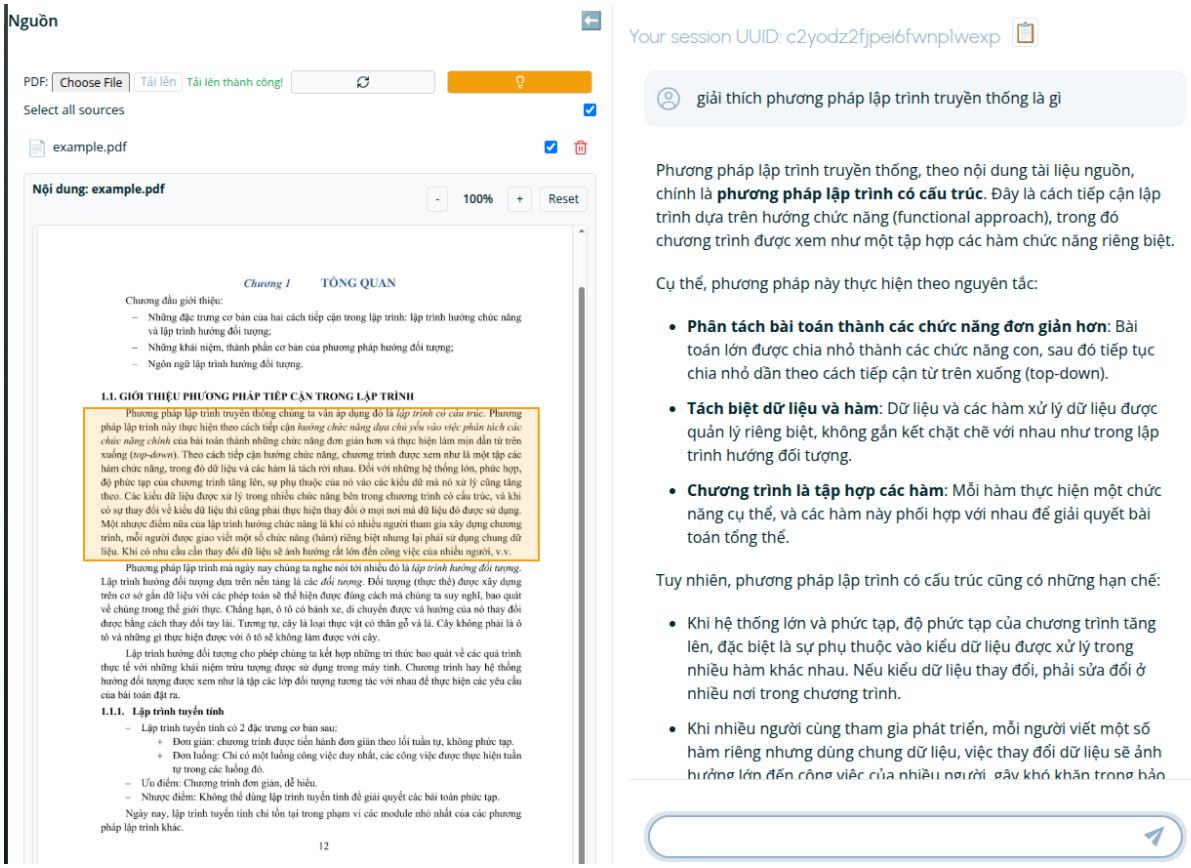


Figure 3.3: Illustration of the document question–answering process

3.2 System Analysis and Design

3.2.1 System Analysis

3.2.1.1 Business Operations of the Interactive Learning System

The system receives learning materials from users in PDF format. Each document is processed to extract textual content, structural elements, headings, and sections, followed by data cleaning and normalization to support downstream learning activities and question generation. The processed content is stored in the database, enabling learners to reuse previously uploaded documents at any time.

Every user who registers for the system is assigned a personal learning profile, which includes personal information, interaction history, learned documents, total study hours, learning streaks, and progress tracking. This profile is used to personalize the overall learning experience.

When a user chooses to study a document, they upload the PDF file, after which the system analyzes the content and automatically generates key topics, a structured table of contents, and a mind map to facilitate comprehension of the document's core concepts.

Learners may request the system to generate a set of questions based on the document using customizable configurations such as the number of questions and cognitive levels (recall, understanding, application, and higher-order thinking). After generation, the system evaluates, filters, and refines the questions to ensure accuracy, clarity, and relevance.

During the learning process, users can ask questions at any time based on the document's content. The system employs an intelligent retrieval mechanism to locate related information segments and provide accurate answers accompanied by detailed explanations, examples, or extended knowledge.

When users complete exercises or answer questions, the system records their results, updates their learning progress, computes scores, and identifies knowledge gaps to recommend appropriate review activities. All learning activities — such as answering questions, viewing mind maps, or tracking study time — are logged to support analytics and personalized recommendations.

Each day a learner logs in and engages with learning tasks, the system tracks study hours and updates the user's streak count. Notifications are provided when a streak is

maintained or broken, encouraging consistency and motivating continued learning.

3.2.1.2 System Requirements

a. Functional Requirements

- **PDF Upload and Content Extraction:** The system shall allow users to upload learning materials in PDF format and support automatic extraction of textual content, structural elements, and relevant metadata for subsequent learning activities and processing tasks.
- **Document-Based Question Generation:** The system shall generate a set of questions from the uploaded document based on user-defined configurations, including the number of questions and cognitive levels (recall, understanding, application, and higher-order thinking).
- **Document Question Answering:** The system shall provide a document-based question answering function, enabling learners to ask questions and receive accurate responses derived from the extracted document content.
- **Learning Progress Tracking:** The system shall store and track the learner's progress, including question history, assessment results, studied documents, and all interactive learning activities.
- **Study Time and Daily Activity Logging:** The system shall record daily study time and learning activities, maintain the learner's study streak, and provide notifications to encourage consistency and motivation.
- **Mind Map Generation:** The system shall support the creation of mind maps from the document content to help learners quickly capture key ideas and visualize the overall knowledge structure.

b. Non-functional Requirements

- **Performance:** The system must process PDF uploads and content extraction within a reasonable time, ensuring that question generation, mind map creation, and document-based question answering operate smoothly without interrupting the user experience.

- **Accuracy:** Generated questions, answers, mind maps, and QA responses must be derived closely from the document content, maintaining correctness and minimizing misinformation or irrelevant outputs.
- **Reliability and Stability:** The system must operate continuously with minimal errors during document processing, progress tracking, and interactive learning activities.
- **Scalability:** The system architecture should support scaling to handle large documents, a high number of concurrent users, and easy integration of additional modules or learning features in the future.
- **Security:** User data including uploaded PDFs, learning progress, study time, and streak information must be securely stored with access control mechanisms and encryption where appropriate.
- **Usability:** The user interface must be intuitive, easy to navigate, and support efficient interaction with key functionalities such as uploading PDFs, answering questions, viewing progress, and exploring mind maps.
- **Responsiveness:** The system must provide real-time or near real-time feedback for actions such as question answering, information retrieval, streak updates, and progress checking, ensuring a seamless interactive learning experience.

c. Functional Hierarchy Diagram Figure 3.4 illustrates the functional hierarchy of the interactive learning system.

3.2.1.3 Modeling with Diagram Types

The system is modeled using four groups of diagrams to comprehensively describe its operational aspects. Activity diagrams illustrate the workflow and business processes of the system; use case diagrams describe user requirements and functional capabilities from the perspective of system actors; sequence diagrams depict temporal interactions among system components; and state machine diagrams represent the behavioral changes of objects under different conditions. Detailed versions of these diagrams are provided in Appendix A - D

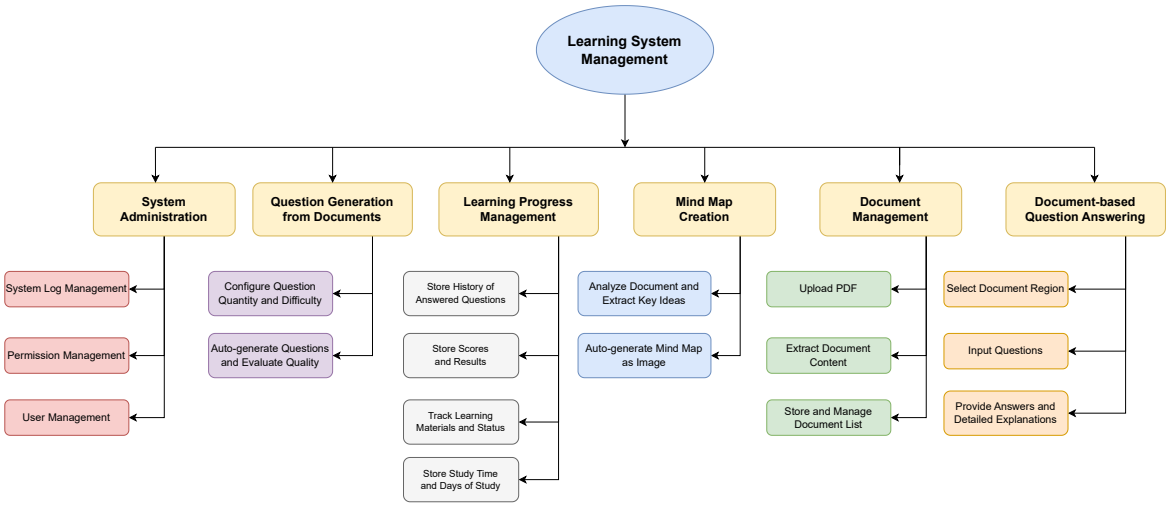


Figure 3.4: Functional hierarchy diagram of the interactive learning system

3.2.2 System Design

The system design is organized into two main parts to clearly present its structural and visual components. The user interface design shows the screens, layouts, and navigation flow of the system, helping readers understand how users interact with the application. The database design describes the tables, fields, and relationships that store and manage the system's data in a structured way. Full and detailed versions of both the interface design and the database schema are presented in Appendix E - F

3.3 Event-Driven Routing Mechanism

Figure 3.5 illustrates the prompt routing mechanism within the multi-module learning system, which is built upon an event-driven architecture. When a user submits a request through the chat interface, the input is forwarded to the Router module, where the system classifies the user's intent and extracts the relevant metadata. Acting as the central orchestrator, the Router analyzes the semantic content of the prompt and determines the type of task the user intends to perform such as summarization, question generation, mind-map creation, or document-based question answering.

After identifying the intent, the Router generates a routing event and publishes it to the Event Bus. The Event Bus functions as an asynchronous communication backbone, enabling each functional module to subscribe to and receive only the event types corresponding to its designated responsibilities. This design ensures that each module handles

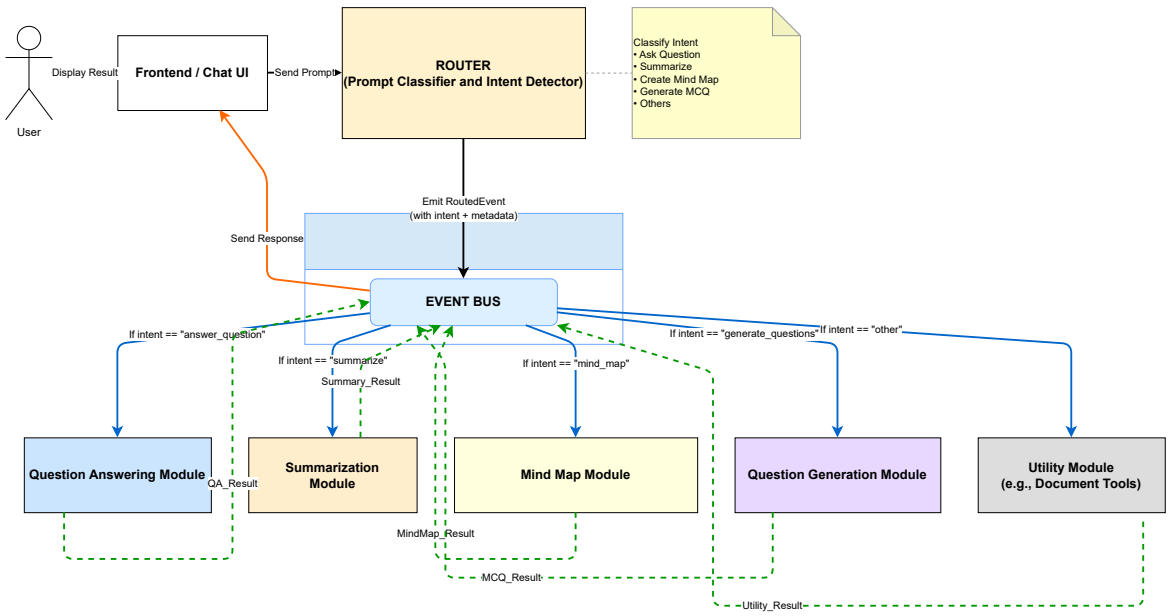


Figure 3.5: Prompt Routing Mechanism for Multi-Module Learning System

only the events relevant to its functionality, maintaining a loosely coupled structure and supporting system scalability.

Specifically, events with intents such as answer question, summarize, mind map, or generate questions are routed to the Question Answering Module, Summarization Module, Mind Map Module, and Question Generation Module, respectively. For auxiliary or miscellaneous tasks, events are dispatched to the Utility Module (e.g., document processing tools). After processing, each module emits a result event (such as QA Result, Summary Result, MindMap Result, or MCQ Result), which is returned to the Event Bus and subsequently delivered to the user interface.

This mechanism enables the system to manage multiple tasks in a modular, maintainable, and extensible manner. Furthermore, the event-driven model allows modules to operate asynchronously and independently, reducing interdependencies between components, enhancing architectural flexibility, and enabling the integration of additional modules in the future without requiring modifications to the core system structure.

3.4 Document Preprocessing and Structuring Module

The document preprocessing pipeline begins with converting input PDF files into structured representations that serve as the foundation for both the question-generation and mind map–generation modules. Figure 3.6 illustrates the main stages of the Document

Preprocessing and Structuring module.

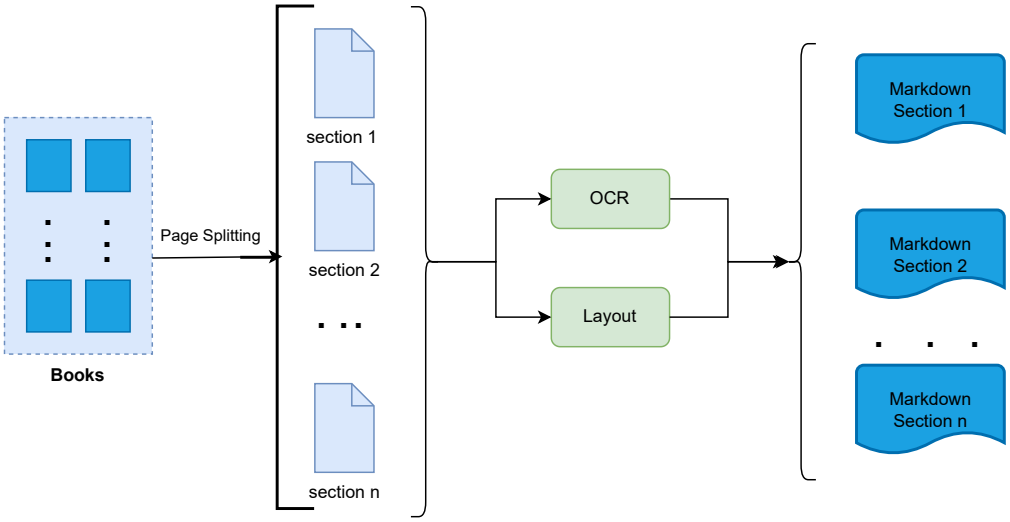


Figure 3.6: Document Processing Pipeline

In the first stage, the system loads the PDF document and rasterizes each page into an image using the *pdf2image* library [28]. This conversion preserves the exact visual layout of the document, enabling more reliable downstream vision-based extraction tasks.Based on these page images, the system uses a VLM to perform OCR, obtaining raw text with high stability across a wide range of document types. In parallel, the system incorporates Docling, a specialized framework for document layout extraction and reconstruction [29]. Docling enables precise identification of semantic components such as headings, paragraphs, tables, figures, and layout structures, producing a hierarchical and semantically rich layout representation that surpasses traditional object-detection–based approaches.

In the final stage, the document is segmented into sections using both textual content and layout information. This segmentation strategy ensures an appropriate level of overlap between semantic units, preserving contextual continuity when processed by VLM in subsequent modules. For each segment, the system combines OCR text with layout metadata to construct a complete, well structured information unit, ready for downstream reasoning tasks such as question generation or mind map synthesis.

3.5 Multiple Choice Question Generation Module

Process begins at the *Prompt* block, where an instructor specifies whether questions should be generated for the entire textbook or for a specific section. If the request targets the full document, the system iterates through all Markdown sections. If the request refers to a single part, the corresponding section is retrieved and passed into the generation pipeline.Before question generation takes place, the textbook is divided into smaller Markdown segments according to its structure. These segments include overlapping regions to preserve contextual continuity and prevent information loss when splitting long documents. This design reduces the context length required by the language model while ensuring that essential information remains available to the agents.

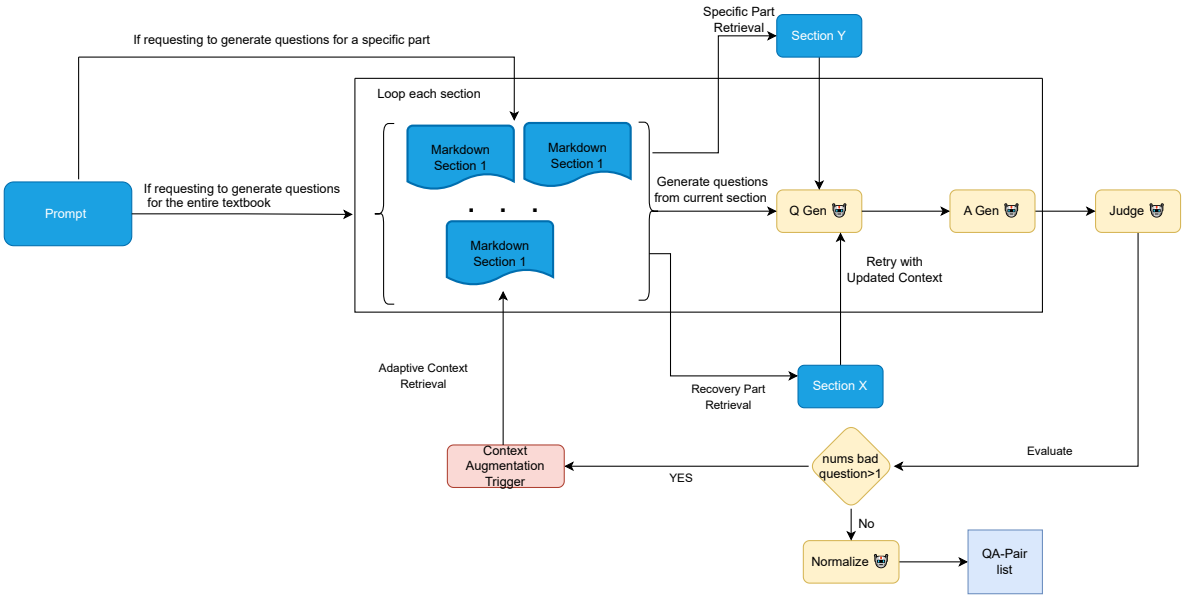


Figure 3.7: Multi-Agent Interactive Chain: Q-Gen (question generation), A-Gen (answer generation), Judge (quality evaluation), iterative refinement, and final normalization.

When a section is selected, it is forwarded to the Q-Gen agent. Q-Gen identifies the key ideas within the segment and generates both the question and the exact relevant passage from which the question is derived. Providing this supporting passage ensures that generated questions remain grounded in the source material and avoids unsupported or ambiguous content. If additional contextual segments (e.g., Section X or Section Y) have been retrieved in earlier cycles, Q-Gen incorporates them to further improve question quality.The outputs from Q-Gen are then processed by the A-Gen agent, which produces the correct answer and, when required, generates distractors. These prelimi-

nary question-answer pairs are evaluated by the Judge agent, which considers accuracy, clarity, difficulty, and alignment with learning objectives. If the number of low-quality questions exceeds a defined threshold, the system activates the *Context Augmentation Trigger*. This component retrieves additional contextual information through two mechanisms: *Adaptive Context Retrieval*, which gathers related neighboring sections, and *Recovery Part Retrieval*, which fetches a specific section needed to repair questions that are incomplete or vague. The updated context is sent back to Q-Gen, forming an iterative improvement loop that continues until all evaluation criteria are satisfied.

After this evaluation phase, the Normalize agent standardizes the format, removes duplicates, assigns metadata such as IDs and difficulty levels, and ensures consistent structure across the final dataset. The resulting output is a polished list of question-answer pairs ready for instructional use. Figure 3.7 demonstrates that MCQ generation is not a simple one step operation but a coordinated multi-agent pipeline. By combining question generation, answer creation, quality evaluation, context augmentation, and output normalization and by integrating section overlap and explicit passage extraction the system consistently produces high-quality, context-grounded questions suitable for programming education.

3.6 Mindmap Module

3.6.1 Document Summarization Module

The Document Summarization module plays a crucial role in generating a condensed knowledge source that serves as the foundation for the mind map generation process. Instead of directly using long textual passages from the original document, the system performs multi-level summarization to extract the most essential content. This ensures that the generated mind maps are concise, intuitive, and accurately reflect the logical structure of the document.

The workflow begins with the *Markdown Sections*, which are standardized content units produced during the preprocessing stage. Each section is processed through two parallel summarization branches to leverage the strengths of both approaches.

First, the *Abstractive Summarization* module generates an interpretative summary, allowing the LLM to reorganize and rephrase information in a natural way while preserving the core meaning. This approach helps highlight key ideas and provides a high-level conceptual abstraction of the content.

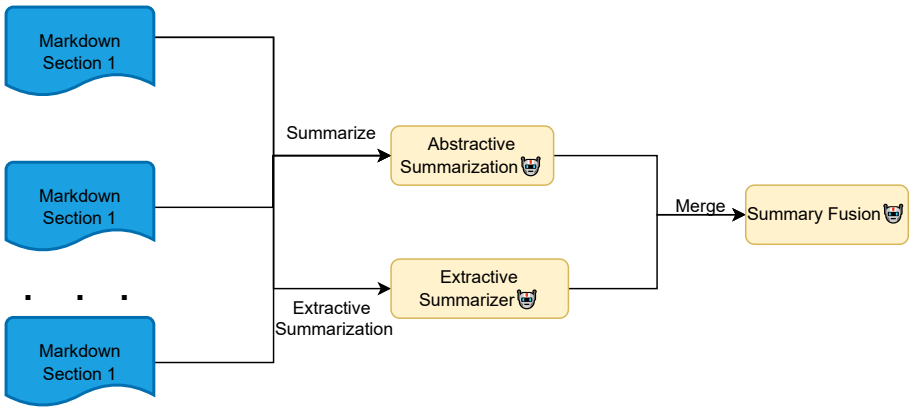


Figure 3.8: Hybrid summarization workflow combining two parallel processing branches: Abstractive Summarization and Extractive Summarization. The outputs from both models are merged through a Summary Fusion step to produce a concise, accurate, and semantically rich final summary.

Second, the *Extractive Summarization* module focuses on selecting the most important sentences or paragraphs directly from the original text. This method maintains the exact wording of the source material, ensuring high factual accuracy and strong fidelity to the original content.

The outputs from these two branches are then passed to the *Summary Fusion* stage. Here, the system integrates both types of summaries by combining their complementary strengths: the conciseness and expressive flexibility of abstractive summarization with the reliability and textual precision of extractive summarization. The fused summary serves as a high-quality input for the mind map generation module in the subsequent phase.By combining these two summarization techniques and unifying their results, the Document Summarization module ensures that the essential concepts, logical structure, and relationships within the document are preserved in a compact and well-organized form. This creates a solid foundation for generating accurate and effective mind maps.

3.6.2 Mindmap Generation Module

The mindmap generation module creates a visual summary of the main ideas in the document. This module first identifies key concepts and then organizes them into a tree like structure. The root represents the main topic, and the branches show related ideas and subtopics.

The purpose of this module is to help users understand the content more easily by showing how the ideas connect. It also supports later steps in the system, such as question generation and summarization, by providing a clear structure of the document. Although the mindmap gives a good overview, its quality still depends on how well the model detects important concepts and relationships in the text.

3.7 Document Question Answering Module

3.7.1 Pre-QA Processing Module

The Data Preprocessing and Structuring module serves as the core preprocessing layer of the entire system, responsible for transforming raw documents such as PDFs, e-books, or scanned pages into a standardized and structured knowledge space. This structured representation is shared across two major downstream components: DQA subsystem and MMG subsystem. The entire pipeline is designed as a multi stage architecture to preserve the original logical structure of the document, enhance the accuracy of information extraction, and create a robust foundation for subsequent reasoning tasks.

The process begins with the input document collection (Books), where the system applies *Page Splitting* to divide the document into a sequence of continuous sections (section 1... section n). This decomposition reduces the processing size of each unit, maintains contextual coherence, and ensures that LVLMs can process the content effectively without exceeding context limitations.

Each section is then passed through two parallel components: *OCR Extraction* and *Layout Reconstruction*. The OCR component converts all character containing regions on the page into raw textual content, reconstructing the linguistic layer of the document. In parallel, the Layout Reconstruction component analyzes the page structure to detect headings, paragraphs, tables, figures, and other structural elements. The outputs of these two steps are merged and standardized into *Markdown Sections* a structured representation that is both easy to analyze and suitable for automated processing.

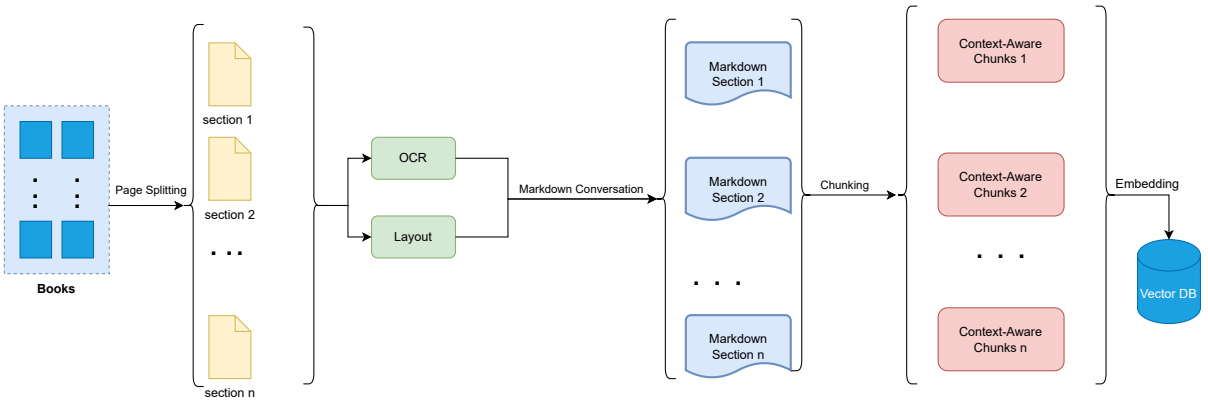


Figure 3.9: Data Preprocessing and Structuring Pipeline: Page Splitting, OCR and Layout Reconstruction, generation of Markdown Sections, context-aware Chunking, and embedding encoding for storage in the Vector Database.

Next, the system applies *Context-Aware Chunking*, a segmentation strategy based on semantic and structural context rather than fixed-length splitting. Each Markdown Section contains two key components: *Page Content* (the raw text extracted by OCR) and *Metadata* (information such as heading levels, component types, page positions, reading order, and layout attributes). Metadata is used to infer document structure (e.g., identifying when a new section begins), while page content helps the system evaluate the semantic continuity between segments. By combining these two sources of signals, the system produces *Context-Aware Chunks* that preserve coherent meaning and ensure that each chunk represents a self-contained unit of knowledge suitable for semantic retrieval tasks.

In the final stage, all Context-Aware Chunks are encoded into vectors using an embedding model and stored in a Vector Database. This representation enables high-accuracy semantic search and forms a reliable foundation for multiple downstream tasks, including document question answering, question generation, summarization, and mind map construction. Through its multi layered processing workflow and its ability to preserve semantic structure, the Data Preprocessing and Structuring module ensures that all input documents are transformed into an organized, consistent, and inference-optimized

knowledge space.

3.7.2 Hybrid Retrieval-Augmented Generation

DQA module adopts a Hybrid RAG mechanism to retrieve and synthesize information from the knowledge base that has been mapped into the embedding space. This module serves as a core component that enables the system to generate accurate, document-grounded responses while minimizing unsupported or hallucinated reasoning.

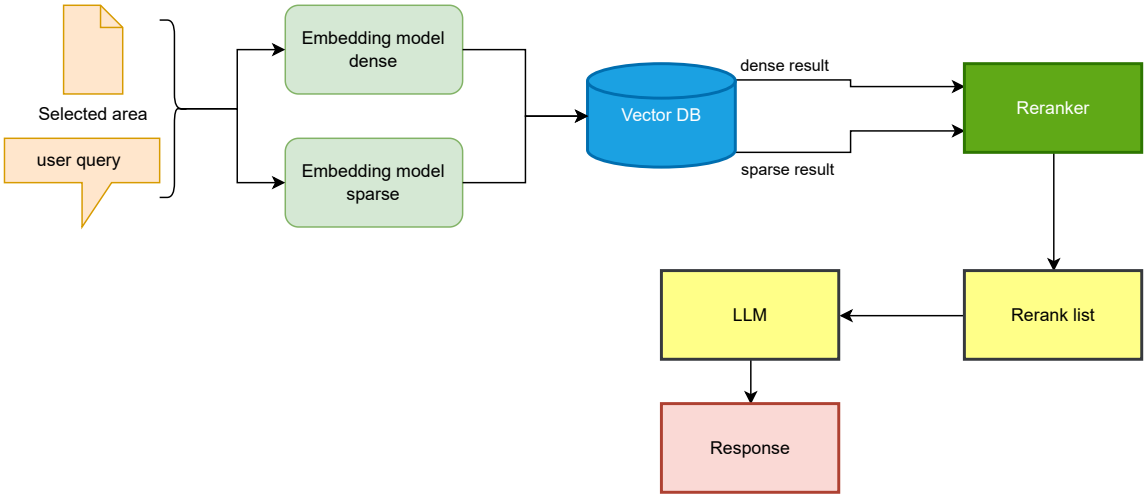


Figure 3.10: The Hybrid RAG workflow for retrieval and information synthesis.

The pipeline begins when the system receives a user query together with an optional user selected content region from the document. Both types of input are encoded simultaneously using two embedding models: a *Dense Embedding Model* and a *Sparse Embedding Model*.

The *Dense Embedding Model* generates continuous vector representations that capture semantic similarity between the query and document segments. In contrast, the *Sparse Embedding Model* emphasizes keyword level matching, making it suitable for queries requiring high lexical precision or domain-specific terminology.

The resulting embeddings are forwarded to the *Vector Database*, where the system performs two parallel retrieval procedures: *dense retrieval* and *sparse retrieval*. Each

retrieval method returns an independent ranked list of candidate passages one based on semantic similarity (dense) and the other based on keyword relevance (sparse).

To further improve retrieval quality, the two candidate lists are merged and passed to the *Reranker* module. This component leverages a specialized scoring model to re-rank all retrieved segments based on deeper contextual relevance, producing a *Reranked List* containing the most pertinent evidence for the user query.

The reranked segments are then provided as contextual input to the LLM. The LLM synthesizes, analyzes, and reasons over these evidence passages to construct the final answer. This process ensures that the generated response is comprehensive, coherent, and grounded in the source document while significantly reducing hallucinations and unsupported inferences.

The complete workflow forms a closed-loop pipeline composed of five stages, beginning with receiving and analyzing the user query, followed by encoding the query using both dense and sparse embedding models. Next, the system retrieves relevant information from the Vector Database, after which the retrieved candidates are re-ranked using the Reranker model. Finally, the system generates the final answer based on the retrieved evidence.

By combining two complementary retrieval mechanisms together with a re-ranking stage, the Hybrid RAG module achieves higher accuracy, improved explainability, and greater reliability. This architecture contributes significantly to the effectiveness of the knowledge driven interactive learning platform.

Chapter 4

Experiments and Results

4.1 Evaluation Methods

4.1.1 Modular Evaluation Framework

The learning system is designed with a modular architecture, allowing for the independent evaluation of each component to clearly determine the role, performance, and contribution of each module to the overall efficiency. The modular evaluation framework encompasses three main functional components: Question Generation, Mind Map Generation, and Question Answering. Each module is assessed based on a specialized set of criteria tailored to the specific task it performs.

Question Generation Evaluation This is tasked with generating objective multiple-choice questions from the input document content. The assessment focuses on three primary dimensions:

1. **Question Clarity and Concept Fidelity:** The ability to accurately identify important concepts and formulate them into clear, unambiguous questions.
2. **Distractor Quality:** The quality of the incorrect options concerning their plausibility, relevance, and ability to reflect common learner misconceptions.
3. **Contextual Alignment:** The degree of contextual alignment with the source material, ensuring the exclusion of extraneous information or misinterpretations.

The evaluation methodology integrates a comparison against a *Direct Prompting* base-

line model and a learner survey involving approximately 20 participants. Question samples are randomly shuffled in the survey to mitigate identification bias.

Summarization Evaluation As discussed in [30], argument-aware evaluation is essential for long-document summarization, the focus is on the retention rate of key ideas (quantified based on the ARC_role metric, covering ARC_fullset), the extent of hallucination mitigation by cross-referencing with the source material, the compression ratio, and the cohesion of the summarized text.

The $\text{ARC}_{\text{fullset}}$ score measures how well the summary S captures the entire argument set A as a whole. It is derived from a Likert-scale judgment $\ell(A, S)$ (1–4), normalized to $[0, 1]$.

$$\text{ARC}_{\text{fullset}}(S) = \Phi(A, S) \tag{4.1}$$

Each salient argument $\{a_i\}$ is evaluated independently. If the argument is fully preserved in the summary, it receives 1. If it is omitted, partially missing, or distorted in the summary, it receives 0.

$$\Phi(a_i, S) = \begin{cases} 1, & \text{if } a_i \text{ is fully preserved in } S, \\ 0, & \text{if } a_i \text{ is partially/fully omitted or distorted in } S. \end{cases} \tag{4.2}$$

The ARC_{role} score aggregates the independent preservation scores across all n salient arguments. It represents the proportion of arguments faithfully preserved in the summary.

$$\text{ARC}_{\text{role}}(S) = \frac{1}{n} \sum_{i=1}^{n} \Phi(a_i, S) \tag{4.3}$$

Question-Answering Evaluation The QA module assesses the system's capability as a learning support tool based on information retrieval and reasoning. The main evaluation criteria include:

- **Retrieval Accuracy:** The ability to accurately retrieve the correct information segment from the source document.

- **Reasoning Foundation:** The capacity for grounded reasoning to formulate logical and well-supported answers.
- **Consistency:** The consistency of responses when handling complex or multi-faceted queries.

The evaluation process is conducted using a set of sample questions constructed from 10 textbooks across various domains to ensure the generality and diversity of the results.

4.1.2 Human Evaluation

To ensure the **ecological validity** of the generated questions, we deployed a **human subject evaluation** (user study) to assess the quality of the output from the QG module.

The evaluation procedure was implemented following the methodology of expert/learner **subjective assessment**, where participants were asked to quantitatively score the multiple-choice questions based on four main criteria:

1. **Clarity and Semantic Coherence:** Assessing the lucidity and quality of the question phrasing.
2. **Fidelity and Content Relevance:** Evaluating the factual accuracy and strict adherence (grounding) to the content of the original textbook material.
3. **Conceptual Coverage:** Determining the extent to which the generated questions comprehensively address the key concepts and important sections of the source document.
4. **Plausibility and Quality of Distractors:** Judging the quality and logical coherence of the incorrect options (distractors), ensuring they reflect common misconceptions without ambiguity.

The quantitative and qualitative feedback obtained from the learners is established as the **ground truth** for comparison against automated scoring results provided by a Large Language Model (LLM-based scoring) and for measuring the **performance improvement** of the proposed system relative to the baseline architecture.

4.1.3 LLM-as-a-Judge

In parallel with the **Human Subject Evaluation**, we utilized LLMs to serve as **automated expert judges** to scale the assessment process and enhance its consistency.

The procedure for the **LLM-based Evaluation** was implemented as follows:

- **Criteria Provision:** The LLM Judge was furnished with a detailed and specific set of evaluation criteria tailored to each module.
- **Quantitative Scoring:** The model was instructed to score each output based on a quantitative scale (e.g., a Likert scale from 1–4 or a point scale from 1–10), aligned with the requirements of the respective module.
- **Reliability Enhancement:** To mitigate the inherent variance of the model, each sample output was scored independently three times, and the final result was obtained by averaging these scores to improve reliability.

The rigorous integration of human evaluation (establishing the ground truth) and automated LLM-based assessment aims to achieve three crucial research objectives:

1. **Correlation Analysis:** To analyze the correlation between the automated scores and the subjective human scores, thereby determining the fidelity of the LLM Judge.
2. **Bias Detection:** To identify instances where the LLM assessment exhibited inaccuracies or biases (divergence) relative to the human ground truth.
3. **Robust Conclusion:** To provide more robust and objective conclusions when comparing the effectiveness between the proposed system and the established baseline architectures.

This combined approach ensures a comprehensive, scientifically rigorous, and objective view of the quality and performance of each system component.

4.2 Collection

The dataset used in this research was built from 10 complete programming textbooks on topics such as Java Object-Oriented Programming (OOP), Data Structures and Algorithms, Programming Principles, Basic Python, and Object-Oriented Programming. Each textbook is between 150 and more than 400 pages long. The textbooks were used in full during the evaluation, instead of being split into small sections or paragraphs as in many traditional methods. Using complete documents allows the model to fully use

context, chapter structure, and the relationships between parts, which better reflects real challenges in long-document processing.

To evaluate the system modules (Question Generation, Summarization, Mind-Map Construction, and Question Answering), the dataset was created using the baseline model GPT-4o and processed through three main steps:

Step 1: Direct Generation of Target Data from Full Documents. Each full textbook was given to GPT-4o together with the specific prompt for each task, as described in Appendix G. Specifically:

- **Question Generation:** GPT-4o generated a set of multiple-choice questions covering the whole textbook.
- **Summarization:** GPT-4o produced summaries at different levels (short, medium, and long).
- **Question Answering:** GPT-4o answered questions based on the full content of the document.

This approach simulates real use cases where the system must process long and complex documents without manual splitting.

Fairness of Baseline Usage. The GPT-4o baseline follows a single-step generation strategy, in which outputs are produced directly from the full document using a single prompt, without intermediate agent decomposition, structured reasoning stages, or explicit pedagogical constraints. In contrast, the proposed system benefits from a multi-agent workflow that introduces role-based reasoning, intermediate representations, and task-specific constraints such as cognitive-level targeting and structured content coverage. **Step 2: Data Review and Cleaning.** The dataset was carefully checked to remove samples that were incorrect, incomplete, misleading, or did not match the original textbook. Summaries with hallucinated information, mind maps with weak structure, duplicated multiple-choice questions, or incorrect QA pairs were corrected or removed.

Step 3: Data Organization for Module-Specific Evaluation. After cleaning, the data was organized into four evaluation sets for Question Generation, Summarization, Mind-Map Construction, and Question Answering. This separation helps ensure fairness and consistency when comparing the proposed system with baseline methods.

The final dataset contains 10 full textbooks, 100 summaries of different lengths, 1000 questions, and 100 question–answer pairs. Using complete documents ensures

that the evaluation reflects the model's real ability to handle long texts and supports the goal of building large-scale learning support systems. Detailed statistics about document length and word count for the 10 textbooks are shown in Table 4.1.

Table 4.1: Statistics of the number of pages and words of the 10 textbooks

Textbook Name	**Number of Pages**	**Number of Words**
Big Data Fundamentals	235	50,542
Cấu trúc dữ liệu và Giải thuật	229	67,255
Intro to R (Vietnamese)	118	39,497
OOP 2013	241	73,277
Deep Learning cơ bản (v2)	301	58,728
Giáo trình Xử lý Ảnh	76	21,498
Nguyên lý Hệ điều hành (2010)	273	60,967
Giáo trình Lập trình C	194	29,189
Cơ sở Dữ liệu (2015)	116	40,096
Quản trị Web	300	77,933

4.3 Results and Analysis

4.3.1 Evaluation of Question Generation Quality

LLMs-Based Evaluation. The results obtained from the automatic evaluation conducted using both Gemini-2.5-Flash and GPT-4o-mini evaluators reveal a consistent advantage of the proposed MLA framework over GPT-4o in terms of both the proportion and stability of positive ratings. For instance, in the *Understanding* criterion, MLA achieves more than 90% of responses in the "Somewhat Agree" and "Agree" categories, whereas GPT-4o remains around 70%, with the remainder distributed across negative ratings. A similar pattern is evident in *Clarity*, where MLA exceeds 95% positive judgments, while GPT-4o exhibits a noticeable portion (approximately 10–20%) of "Somewhat Disagree" ratings, indicating issues related to ambiguity or inconsistent phrasing.

In terms of overall *Quality*, MLA maintains above 80% favorable ratings, contrasting with GPT-4o, which shows nearly 30% negative evaluations. The *Difficulty* distribution further highlights MLA's ability to regulate question complexity, with more than 70% of responses falling within the two positive categories, whereas GPT-4o displays a less controlled distribution, producing questions that are either overly simple or

unnecessarily complex. For the *Cognitive Level* criterion, MLA demonstrates superior performance by generating questions that require higher-order reasoning, achieving over 75% positive ratings compared to GPT-4o's 55–60%. Finally, in the *Engagement* dimension, MLA again outperforms the baseline, reaching more than 80% positive responses while GPT-4o retains 20–30% negative assessments.

Overall, the results in Figure 4.1 illustrate that the MLA architecture delivers question sets with substantially higher clarity, conceptual depth, and engagement. This improvement stems from the system's multi-agent workflow, in which retrieval, generation, critique, and refinement agents iteratively collaborate to correct errors, calibrate difficulty, and enhance linguistic coherence. In contrast, the single-step generation approach of GPT-4o lacks such layered verification, leading to greater variability and lower reliability. These findings underscore the effectiveness of the multi-agent design in producing high-quality multiple-choice questions aligned with pedagogical standards. Figure 4.1 presents a comparative analysis of question quality between the proposed MLA system and the baseline GPT-4o model across six evaluation criteria: *Understanding*, *Clarity*, *Quality*, *Difficulty*, *Cognitive Level*, and *Engagement*.

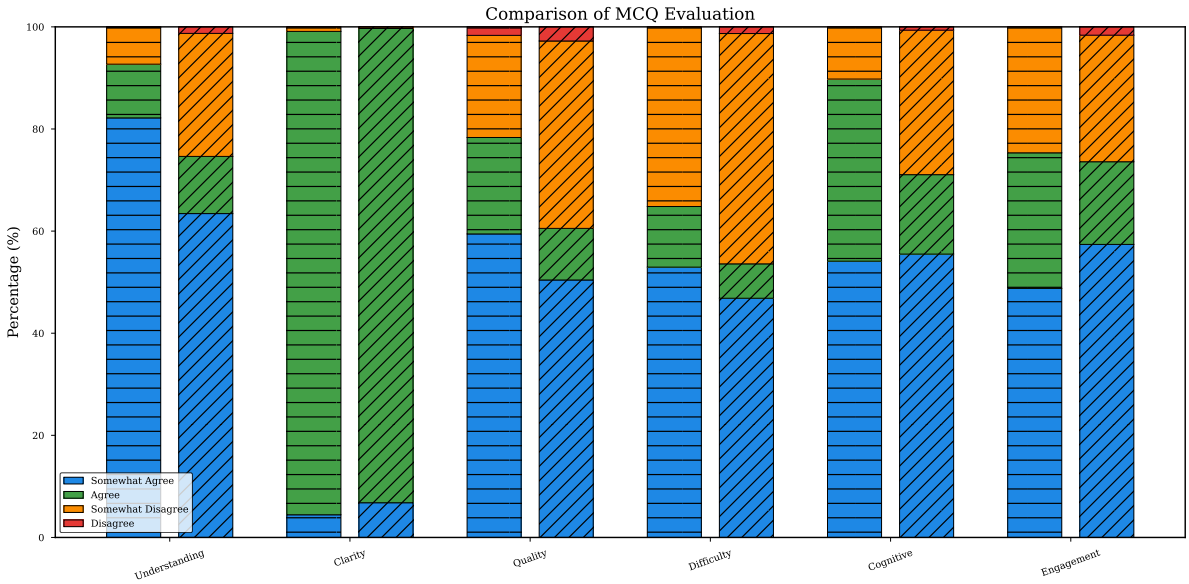


Figure 4.1: Comparison of question quality ratings across evaluation criteria. Bars corresponding to the MLA system use a horizontal hatch pattern ("–"), while bars representing the GPT-4o baseline use a diagonal hatch pattern ("//"), enabling clear visual distinction between the two systems.

Human Evaluation Summary. The results from 20 participants are shown in four pie charts. These charts represent four criteria: clarity of the question, coverage of the source content, quality of the distractors, and overall content appropriateness. The rating scale uses four colors: red (low), orange (below expectation), green (good), and purple (excellent). Most people chose green or purple, which means that the questions were generally well accepted.

For *clarity*, about 95% of participants gave high scores, and around 70% chose the highest score. This shows that the system can produce clear and easy-to-read questions. This good result comes from the prompts that guide the model on how to write a clear question and from the refinement steps that remove unclear text.

For *coverage*, about 90% of participants said that the questions matched the main ideas of the source text. About 65% gave the highest score. This means that the retrieval step and the context selection in the pipeline are working well. A few lower scores may happen when a question focuses on less important details instead of the main idea.

The *distractor quality* criterion shows more variation. Around 75% gave good or excellent scores, but about 25% chose average or low scores. This makes distractor quality the weakest criterion. Making good distractors is difficult. A distractor must be wrong but still make sense. It should not be too easy or too different from the topic. The current pipeline does not have a special step to check or improve distractors. Also, the prompts for distractors are not strict enough, so the model sometimes creates distractors that are weak or not related. This explains the lower ratings.

For *content appropriateness*, about 80% of participants said that the questions fit well with the source content. The scores here are more spread out. This means that some questions did not focus on the main idea or were not at the right difficulty level.

In summary, the system works very well in clarity and content coverage. The other two criteria, distractor quality and content appropriateness, are still good but less stable. The main reason is that the pipeline does not have a step to check distractors carefully, and the prompts do not give strong rules for creating good distractors. Improving these parts may help increase the overall quality of the generated questions. As shown in Figure 4.2, the four pie charts give a clear view of how people rated the generated questions. These charts help us understand the strengths and weaknesses of the system across the four criteria. The visual results support the written analysis and make it easier to see why clarity and coverage received higher scores, while distractor quality and content appropriateness showed more variation. The figure also highlights the parts of the pipeline

that still need improvement, especially the creation and checking of distractors.

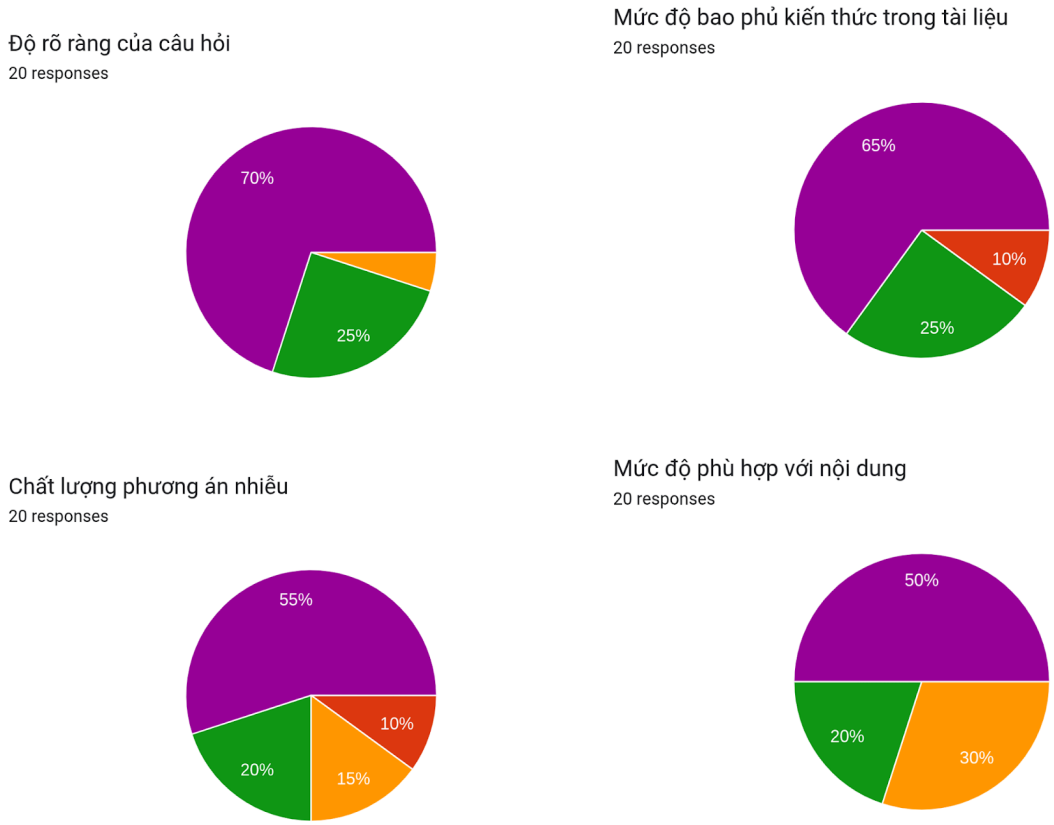


Figure 4.2: Human Evaluation of the MLA System. The rating scale is color-coded from lowest to highest as follows: red (low), orange (below expectation), green (good), and purple (excellent).

4.3.2 Summary of Results

The table shows two ARC metrics for the summaries: *ARC_fullset* and *ARC_role*. These two metrics measure different aspects of summary quality. *ARC_ fullset* tells us whether the summary includes all important ideas from the original text, while *ARC_role* tells us whether the summary keeps the correct roles and relationships between ideas.

Most datasets have an *ARC_ fullset* score of 1.0. For example, the datasets *bigdata*, *csdl*, *ctdl*, *hdh*, *oop*, *r*, *web*, and *xulyanh* all reach 1.0. This means the summaries for these datasets cover almost all the key points from the text. Only one dataset, *giaotrinhc*, has a lower score of 0.67, which means the summary misses some important ideas. This dataset may be more complex or contain more detailed sections, which makes it harder for the system to include everything.

In contrast, the *ARC_role* scores are much lower and show more variation. For example, *bigdata* has a high role score of 0.4465, and *xulyanh* also has a high score of 0.3725. However, several datasets have very low scores: *giaotrinhc* has 0.065, *web* has 0.0805, and *ctdl* has 0.1085. These low scores show that the summaries include the important ideas but do not keep the correct relationships between them. For instance, a definition and an example may be mixed together, or the summary may change the order of ideas in a way that makes them less clear.

This difference between the two metrics occurs because the system is better at collecting important ideas than at preserving the structure of the text. The pipeline does not include a dedicated step for checking roles or relationships, so the model sometimes loses these connections when shortening the content. In addition, the prompts do not give strict instructions for keeping roles, which leads the model to focus more on listing ideas rather than maintaining logical structure.

Overall, the summaries cover the content well but often lose the correct relationships between ideas. This explains why *ARC_ fullset* is high while *ARC_role* is low in many datasets. Table 4.2 presents the ARC scores computed for each dataset.

4.4 Discussions

4.4.1 LLMs Performance in Long Context Retrieval

Figure 4.3 illustrates the relationship between the **Accuracy (%)** of the LLMs in answering questions and the **Position of the Correct Answer** within the input document. This

Dataset	ARC_fullset	ARC_role
bigdata	1	0.4465
csdl	1	0.127
ctdl	1	0.1085
giaotrinhc	0.67	0.065
hdh	1	0.268
oop	1	0.3438
r	1	0.117
web	1	0.0805
xulyanh	1	0.3725

Table 4.2: ARC metrics for summary evaluation of the MLA system across datasets

experiment was designed to evaluate the LLMs retrieval capability across long context documents. The results clearly demonstrate the phenomenon known as **"Lost-in-the-Middle"** in long-context processing [31]. The model's accuracy peaked at approximately 65% when the answer was located at the **beginning (1st)** and the **end (20th)** of the document. This suggests that the LLM tends to best recall and retrieve information situated at the extremities of its input context. Conversely, accuracy suffered a severe drop as the answer position shifted towards the middle of the document, specifically plummeting to about $\mathbf{40\%}$ when the answer was located at the $\mathbf{10th}$ position, only recovering gradually toward the end (positions $\mathbf{15th}$ and $\mathbf{20th}$). This pattern indicates that the model's attention mechanism does not distribute uniformly across the entire context [31]. As input length increases, the model struggles to maintain high attention weights for tokens located in the middle region, often leading to the critical information being "lost." Therefore, achieving optimal accuracy in RAG or long-document summarization applications necessitates applying pre-processing techniques to ensure critical information is strategically positioned at the start or end of the LLMs context window.

4.4.2 Biases in LLM-Generated Multiple-Choice Questions

Our analysis shows that the model still produces several types of bias when generating multiple-choice questions. These issues appear in both the correct answers and the distractors. Similar forms of bias have also been observed in recent research on the robustness of LLMs in multiple-choice settings [32].

One clear problem is that the model tends to follow hidden patterns when creating

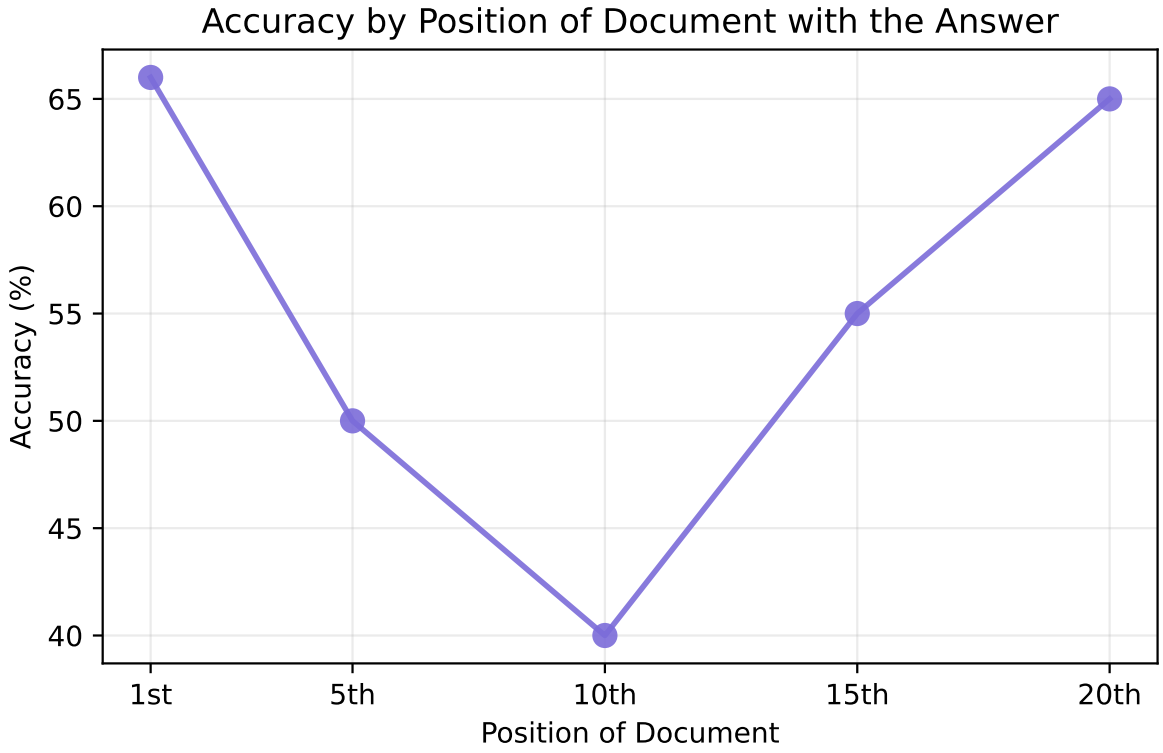


Figure 4.3: LLMs processing long context

options. For example, the correct answer often has a more formal or precise style, while distractors may be shorter, less detailed, or less relevant. This makes the question easier to guess and reduces the overall challenge of the item. These patterns are not intentional, but they come from the model's preference for certain tokens and writing styles.

The study by Zheng et al. shows that LLMs can be strongly influenced by the position or ID of the answer option. This behavior, called selection bias, means the model may prefer an option simply because it appears as "A" or "B," not because it is correct. This bias is caused by token-level preferences learned during training, and it can make the model sensitive to small changes in option order. Our generated questions show similar signs: distractors are sometimes too weak, while the correct answer becomes easier to identify because of consistent linguistic or positional patterns.[32].

These observations suggest that LLMs do not always treat each option fairly when generating or evaluating MCQs. As a result, questions may look well-formed at first sight but still contain predictable structures that lower their quality. This supports the idea that additional control steps or debiasing methods are needed when using LLMs for educational question generation. The PriDe method proposed by Zheng et al. is an example of how debiasing at inference time can help reduce this problem, but our

findings show that more work is still needed to create balanced and reliable distractors in fully automated pipelines.[32].

4.4.3 Limitations in Distractor Generation Pipeline

The distractor generation pipeline still faces several challenges that limit its overall effectiveness. One issue is that the system does not always understand the subtle differences between concepts in the source material. Because of this, the model may produce distractors that do not match the level of detail or type of information found in the question. This mismatch can make the distractors feel disconnected from the main idea.

Another challenge is that the model sometimes focuses too much on the correct answer when forming the distractors. As a result, the distractors may become overly simple or lack the specific context needed to appear realistic. This happens because the pipeline does not guide the model to explore a wider range of possible misunderstandings or common mistakes that real learners might make.

There is also a limitation in how the pipeline handles variation. Many distractors follow similar structures or rely on repeated wording, which reduces diversity across questions. When distractors look too similar to each other, the question loses depth because it does not test different aspects of the concept.

Finally, the current pipeline does not include a mechanism to adjust the difficulty of distractors. Some distractors become too general, while others may introduce information that is too complex or unrelated. Without a balancing step, the system cannot control how challenging the distractors should be for different types of questions.

These limitations show that creating effective distractors requires more than simply generating alternative options. It requires deeper understanding of the content, variation in reasoning patterns, and better control over the type and difficulty of each distractor.

4.4.4 Interpretation of ARC Metrics for Summary Evaluation

The ARC metrics give a deeper understanding of how the system handles summarization. While the summaries often capture many important ideas, the ARC results show that the model does not always keep the original structure of the text. This difference suggests that identifying key content and understanding the relationships between ideas require different abilities.

The high completeness scores indicate that the model is good at finding and col-

lecting important information. However, the lower scores for role consistency show that the model has difficulty keeping the logical connections between ideas. When the text is shortened, the model may mix different types of information or change the order in a way that makes the meaning less clear. This happens because the summarization process focuses mainly on selecting key points rather than preserving how these points relate to each other in the original document.

Another factor is that the current pipeline does not include a step to check or correct the roles of ideas in the summary. Without this, the model may remove or simplify information that is important for understanding the structure of the content. The prompts also give more guidance on what to include than on how to organize it, so the summary can become a list of ideas instead of a well-formed representation of the text.

These observations show that the summarization system is strong at identifying what is important but still needs improvement in keeping how the ideas connect. This affects the overall clarity of the summary and shows where future development should focus.

4.4.5 Human vs LLM Evaluation Divergence

When comparing the scores given by human evaluators with the scores produced by the model itself, a clear difference can be observed. This difference appears mainly in the quality of the distractors. Human evaluators often give lower scores than the model, which suggests that the model shows a form of self-evaluation bias. In other words, the model tends to rate its own output more positively than people do.

This gap shows that an LLM cannot fully replace human judgment when evaluating multiple-choice questions. While the model can check basic structure and content, humans notice many details that the model often overlooks, such as distractors that are too easy, not logical, or not suitable for the question. These small issues can change how difficult and how fair the question is.

Because of this, the difference between human and model evaluations highlights the importance of including human review in the question-generation process. Human feedback helps identify weaknesses that the model cannot detect and ensures that the final questions are reliable and appropriate for educational use.

Conclusions and Future Work

This thesis presented a modular, multi-agent learning support system built on Large Language Models. The system was designed to help learners understand documents more effectively through several key functions, including automatic question generation, document question–answering, summarization, and mindmap creation. Each module focuses on a specific task, and together they form a complete workflow that supports learning in a clear and structured way.

The question generation module uses a two-agent pipeline, where a Generator creates questions based on the document content and learning goals, and a Reviewer checks the clarity, coverage, and cognitive level. This design helps produce questions that are more consistent and aligned with educational needs. The question–answering module supports learners by retrieving relevant information from the document and giving clear answers. The summarization and mindmap modules help learners see the main ideas and the structure of the document, making it easier to study and review.

Several experiments were conducted to examine the strengths and limitations of the system. The ARC metrics showed that the summaries usually captured the important content but sometimes missed the relationships between ideas. Human evaluation revealed differences between model judgments and user expectations, especially in distractor quality. These findings highlight how each module performs and show where improvements are needed. Overall, the results demonstrate that a multi-agent design can provide better clarity, structure, and learning support compared to a single-model approach.

The system developed in this thesis shows that Large Language Models can play an important role in educational technology when combined with careful design and evaluation. The framework provides a foundation for future systems that aim to support personalized learning, improve assessment quality, and offer meaningful feedback to learners.

Although the system shows strong results, there are several directions for improvement. One important area is enhancing the quality of distractors by adding a checking module or applying debiasing methods such as PriDe to reduce option bias and make the distractors more realistic[32]. Another direction is improving the consistency of summarization by introducing an agent that verifies concept roles, which would help increase ARC_role scores and preserve the logical structure of the document. The system could also benefit from personalized learning features that track learner progress, mastery levels, and common mistakes, allowing the system to adapt question difficulty and provide more targeted feedback. In addition, combining human feedback with LLM evaluation through a feedback loop would help the model learn from user ratings and continuously improve the quality of both questions and answers. Finally, conducting real-user studies would make it possible to measure the system's impact in real learning environments and gather valuable data for further refinement. Together, these directions can guide the future development of a smarter, more flexible, and more effective learning support system based on Large Language Models.

References

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30. Curran Associates, Inc., 2017.

[2] J. Liu, D. Shen, Y. Zhang, B. Dolan, L. Carin, and W. Chen, "What makes good in-context examples for gpt-3?" in *Proceedings of Deep Learning Inside Out (DeeLIO 2022): The 3rd Workshop on Knowledge Extraction and Integration for Deep Learning Architectures*, 2022.

[3] H. W. Chung, L. Hou, S. Longpre, B. Zoph, Y. Tay, W. Fedus, Y. Li, X. Wang, M. Dehghani, S. Brahma, A. Webson, S. S. Gu, Z. Dai, M. Suzgun, X. Chen, A. Chowdhery, A. Castro-Ros, M. Pellat, K. Robinson, D. Valter, S. Narang, G. Mishra, A. Yu, V. Zhao, Y. Huang, A. Dai, H. Yu, S. Petrov, E. H. Chi, J. Dean, J. Devlin, A. Roberts, D. Zhou, Q. V. Le, and J. Wei, "Scaling instruction-finetuned language models," 2022.

[4] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, E. Grave, and G. Lample, "Llama: Open and efficient foundation language models," 2023.

[5] NVIDIA, "What are vision language models?" https://www.nvidia.com/en-us/glossary/vision-language-models/, 2023, accessed: 2025-11-24.

[6] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Kulkarni, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive nlp tasks," *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474, 2020.

[7] S. Chen, Y. Liu, W. Han, W. Zhang, and T. Liu, "A survey on llm-based multi-agent systems: Recent advances and new frontiers in application," *arXiv preprint arXiv:2412.17481*, 2024. [Online]. Available: https://arxiv.org/abs/2412.17481

[8] Wikipedia contributors, "Intelligent tutoring system," https://en.wikipedia.org/wiki/Intelligent_tutoring_system, 2024, accessed: 2025-05-07.

[9] S. Wang, T. Liu, J. Tan, Z. Liu, S. Zhang, Y. Yang, Y. Yu, X. Zhang, N. Sun, R. Xu *et al.*, "Large language models for education: A survey and outlook," *arXiv preprint arXiv:2403.18105*, 2024. [Online]. Available: https://arxiv.org/abs/2403.18105

[10] Anonymous, "Embracing imperfection: Simulating students with diverse cognitive levels using llm-based agents," *arXiv preprint arXiv:https://arxiv.org/abs/2505.19997*, 2024, uRL and details to be updated when final bibliographic information is available.

[11] Tesseract OCR Community, "Tesseract ocr engine," https://github.com/tesseract-ocr, 2024, accessed: 2024.

[12] P. Zhang, Y. Xu, Z. Cheng, S. Pu, J. Lu, L. Qiao, Y. Niu, and F. Wu, "Trie: End-to-end text reading and information extraction for document understanding," in *Proceedings of the 28th ACM International Conference on Multimedia (MM '20)*, 2020. [Online]. Available: https://arxiv.org/abs/2005.13118

[13] Y. Xu, M. Li, L. Cui, S. Huang, F. Wei, and M. Zhou, "Layoutlm: Pre-training of text and layout for document image understanding," in *Proceedings of KDD*, 2019.

[14] T. H. Hong, H. Park, S. Kim, S. Yoon, M. Seo, and B.-T. Lee, "Bros: A pre-trained neural network for understanding text in structured documents," in *Proceedings of AAAI*, 2021.

[15] G. Jaume, H. K. Ekenel, and J.-P. Thiran, "Funsd: A dataset for form understanding in noisy scanned documents," in *ICDAR Workshop*, 2019.

[16] S. Park, S. Shin, B. Lee, J. Lee, J. Surh, M. Kim, C. Park, A. Lee, and J. Lee, "Cord: A consolidated receipt dataset for post-ocr parsing," in *ACM MM*, 2019.

[17] M. S. Nacson, A. Aberdam, R. Ganz, E. Ben Avraham, A. Golts, Y. Kittenplon, S. Mazor, and R. Litman, "Docvlm: Make your vlm an efficient reader," *arXiv preprint arXiv:2412.08746*, 2024. [Online]. Available: https://arxiv.org/abs/2412.08746

[18] M. Seo, A. Kembhavi, A. Farhadi, and H. Hajishirzi, "Bidirectional attention flow for machine comprehension," in *International Conference on Learning*

Representations (ICLR), 2017. [Online]. Available: https://arxiv.org/abs/1611.01603

[19] M. Wang, Y. Zhang, and Y. Liu, "Gated self-matching networks for reading comprehension," in *Proceedings of ACL 2017*, 2017. [Online]. Available: https://aclanthology.org/P17-1171/

[20] E. Choi, S. Biswal, B. Malin, J. Duke, W. F. Stewart, and J. Sun, "Generating multi-label discrete patient records using generative adversarial networks," in *Proceedings of the Machine Learning for Healthcare Conference (MLHC)*, 2017.

[21] K. Sun, D. Yu, D. Yu, and C. Cardie, "Improving machine reading comprehension with general reading strategies," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2019.

[22] M. Mathew, D. Karatzas, and C. Jawahar, "Docvqa: A dataset for vqa on document images," in *IEEE Winter Conference on Applications of Computer Vision (WACV)*, 2021, pp. 2200–2209. [Online]. Available: https://arxiv.org/abs/2007.00398

[23] R. Tanaka, K. Nishida, and S. Yoshida, "Visualmrc: Machine reading comprehension on document images," in *Thirty-Fifth AAAI Conference on Artificial Intelligence (AAAI-21)*, 2021. [Online]. Available: https://arxiv.org/abs/2101.11272

[24] T. McDonald, B. Tsan, A. Saini, J. Ordonez, L. Gutierrez, P. Nguyen, B. Mason, and B. Ng, "Detect, retrieve, comprehend: A flexible framework for zero-shot document-level question answering," *arXiv preprint arXiv:2210.01959*, 2022. [Online]. Available: https://arxiv.org/abs/2210.01959

[25] S. Guo, L. Liao, C. Li, and T.-S. Chua, "A survey on neural question generation: Methods, applications, and prospects," *arXiv preprint arXiv:2402.18267*, 2024. [Online]. Available: https://arxiv.org/abs/2402.18267

[26] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, "Exploring the limits of transfer learning with a unified text-to-text transformer," *arXiv preprint arXiv:1910.10683*, 2019. [Online]. Available: https://arxiv.org/abs/1910.10683

[27] A. K. Bhowmick, A. Jagmohan, A. Vempaty, P. Dey, L. Hall, J. Hartman, R. Kokku, and H. Maheshwari, "Automating question generation from

educational text," *arXiv preprint arXiv:2309.15004*, 2023. [Online]. Available: https://arxiv.org/pdf/2309.15004

[28] E. Belval, "pdf2image documentation," https://pdf2image.readthedocs.io/, 2022, accessed: 2025-11-25.

[29] C. Auer, M. Lysak, A. Nassar, M. Dolfi, N. Livathinos, P. Vagenas, C. B. Ramis, M. Omenetti, F. Lindlbauer, K. Dinkla, L. Mishra, Y. Kim, S. Gupta, R. Teixeira de Lima, V. Weber, L. Morin, I. Meijer, V. Kuropiatnyk, and P. W. J. Staar, "Docling technical report," *arXiv preprint arXiv:2408.09869*, 2024. [Online]. Available: https://arxiv.org/abs/2408.09869

[30] M. Elaraby and D. Litman, "Arc: Argument representation and coverage analysis for zero-shot long document summarization with instruction following llms," *arXiv preprint arXiv:2505.23654*, 2025. [Online]. Available: https://arxiv.org/abs/2505.23654

[31] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the middle: How language models use long contexts," *arXiv preprint arXiv:2307.03172*, 2023. [Online]. Available: https://arxiv.org/abs/2307.03172

[32] C. Zheng, H. Zhou, F. Meng, J. Zhou, and M. Huang, "Large language models are not robust multiple choice selectors," *arXiv preprint arXiv:2306.XXXXX*, 2023.

Appendix A

Activity Diagram

This appendix presents the activity diagrams that describe the detailed workflow between the learner and the system in four main processes. Each diagram shows the interaction steps, the system checks, and the feedback returned to the user. These diagrams help provide a clear and visual explanation of how each function works.

The processes include:

(1) **Uploading documents** – shows how the learner selects a PDF file, how the system checks whether the file is valid, and how the content is extracted and stored in the vector database.

(2) **Generating questions from a document** – shows how the learner chooses a document, sets the number and difficulty of questions, and how the system validates the settings and analyzes the document to generate questions automatically.

(3) **Document question–answering** – shows how the learner selects a content area, how the system verifies the selected region, retrieves the related context, and produces an answer.

(4) **Generating a mindmap** – shows how the system analyzes the document, extracts key ideas, and creates a complete mindmap.

The corresponding diagrams are shown from Figure A.1 to Figure A.3, providing a visual and easy-to-follow overview of all workflows.

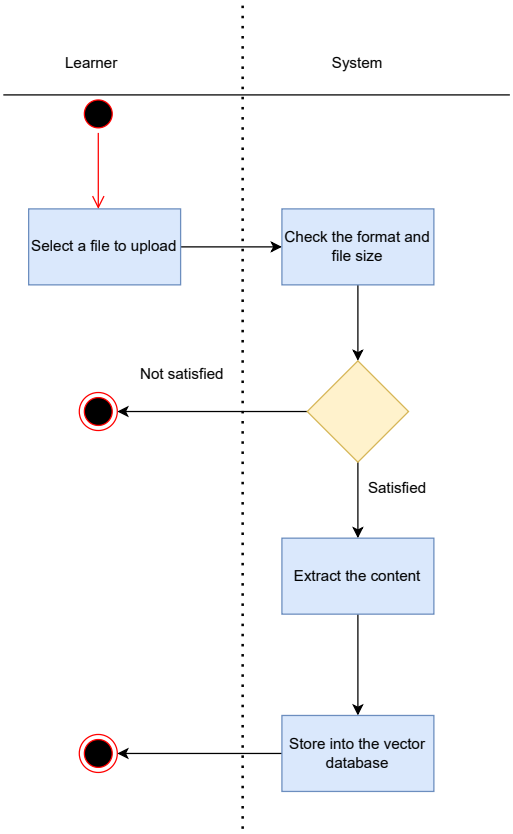


Figure 1: Activity Diagram of PDF Upload

Figure A.1: Activity diagram of the process

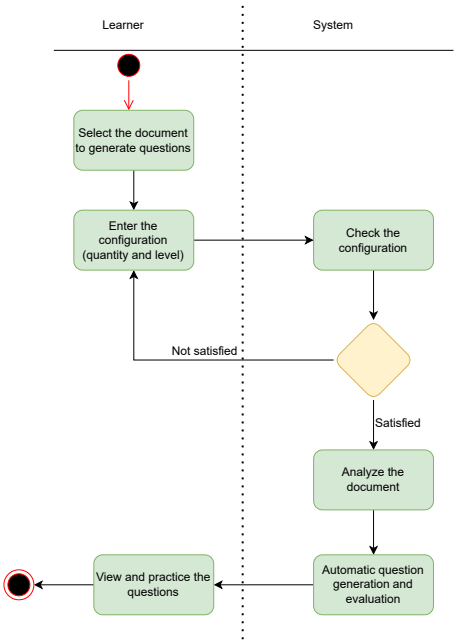


Figure 2: Activity Diagram of the Question Generation Process

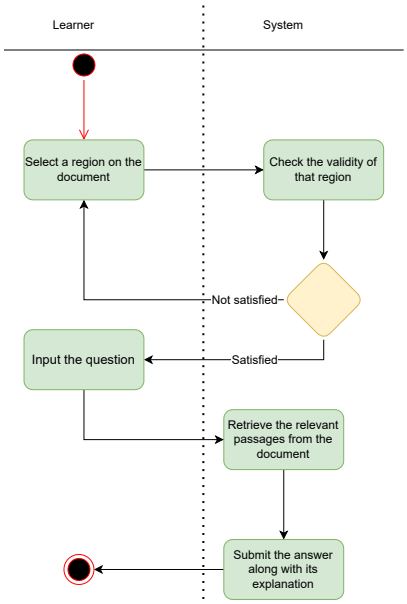


Figure 3: Activity Diagram of Document Question–Answering

Figure A.2: Continuation of the activity diagram

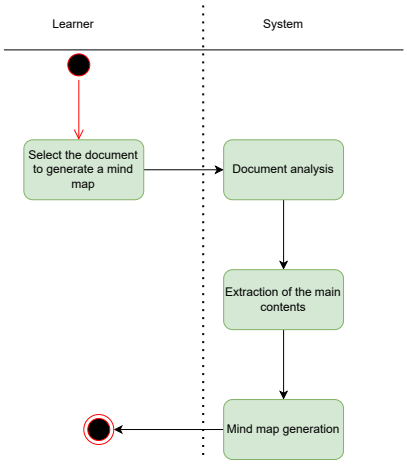


Figure 4: Activity Diagram of the Mind Map Process

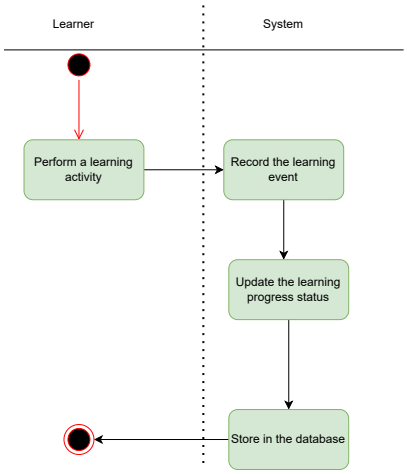


Figure 5: Diagram of the Learning Progress

Figure A.3: Final part of the activity diagram

Appendix B

Usecase Diagram

This appendix presents a set of use cases that describe how learners interact with the system during their study activities. Each use case represents an important function that helps the system support learners in different tasks. The implemented use cases include:

(1) **Uploading documents** – allows learners to upload PDF files so the system can analyze and extract the content. This information is then used for question generation, question–answering, and mindmap creation.

(2) **Practicing questions** – helps learners review knowledge through a set of questions automatically generated from the documents. The system also scores the answers, gives explanations, and records the learner's results.

(3) **Document question–answering** – allows learners to ask questions related to the document content and receive accurate answers using a Retrieval-Augmented Generation (RAG) mechanism.

(4) **Creating a mindmap** – helps learners understand the structure of the document by automatically generating a mindmap from key topics and relationships.

(5) **Viewing learning progress** – allows learners to track their study history, progress level, practice sessions, and various indicators related to their learning habits.

An overview of the relationships between the actors and these functions is shown in Figure B.1. This diagram provides a clear and visual illustration of the interaction structure of the entire system.



Figure 1: Use Case Diagram of the Interactive Learning System

Figure B.1: Overview use case diagram of the system.

Appendix C

Sequence Diagram

This appendix presents the sequence diagrams that describe in detail the interaction process between the learner and the system components in the core functions. Each diagram shows the order of messages exchanged between the actors, how the system handles each request, and how the results are returned. These diagrams help explain the operational workflow of each module over time.

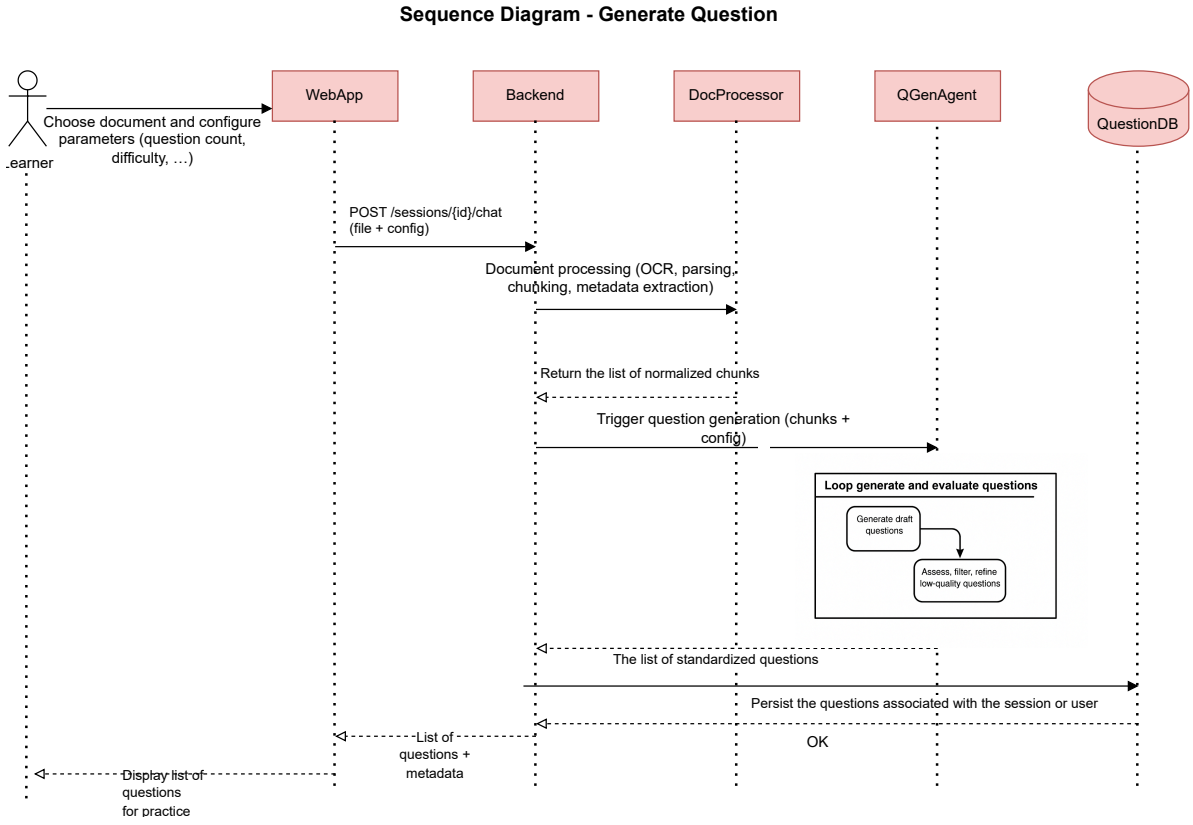
The modeled processes include:

(1) **Question practice process** – describes how the learner selects a practice session, receives a list of questions, submits answers, gets evaluation results, and views overall learning progress.

(2) **Mindmap creation process** – describes how the system checks the document cache, processes the document (OCR, parsing, chunking), summarizes the content, and generates the mindmap structure.

(3) **Question generation process** – describes how the learner selects a document, how the system processes the document, extracts chunks, activates the automatic question generation mechanism, and returns the final list of questions.

The corresponding diagrams are shown from Figure C.1 to Figure C.2, providing a clear and visual illustration of the step-by-step operations of the system.



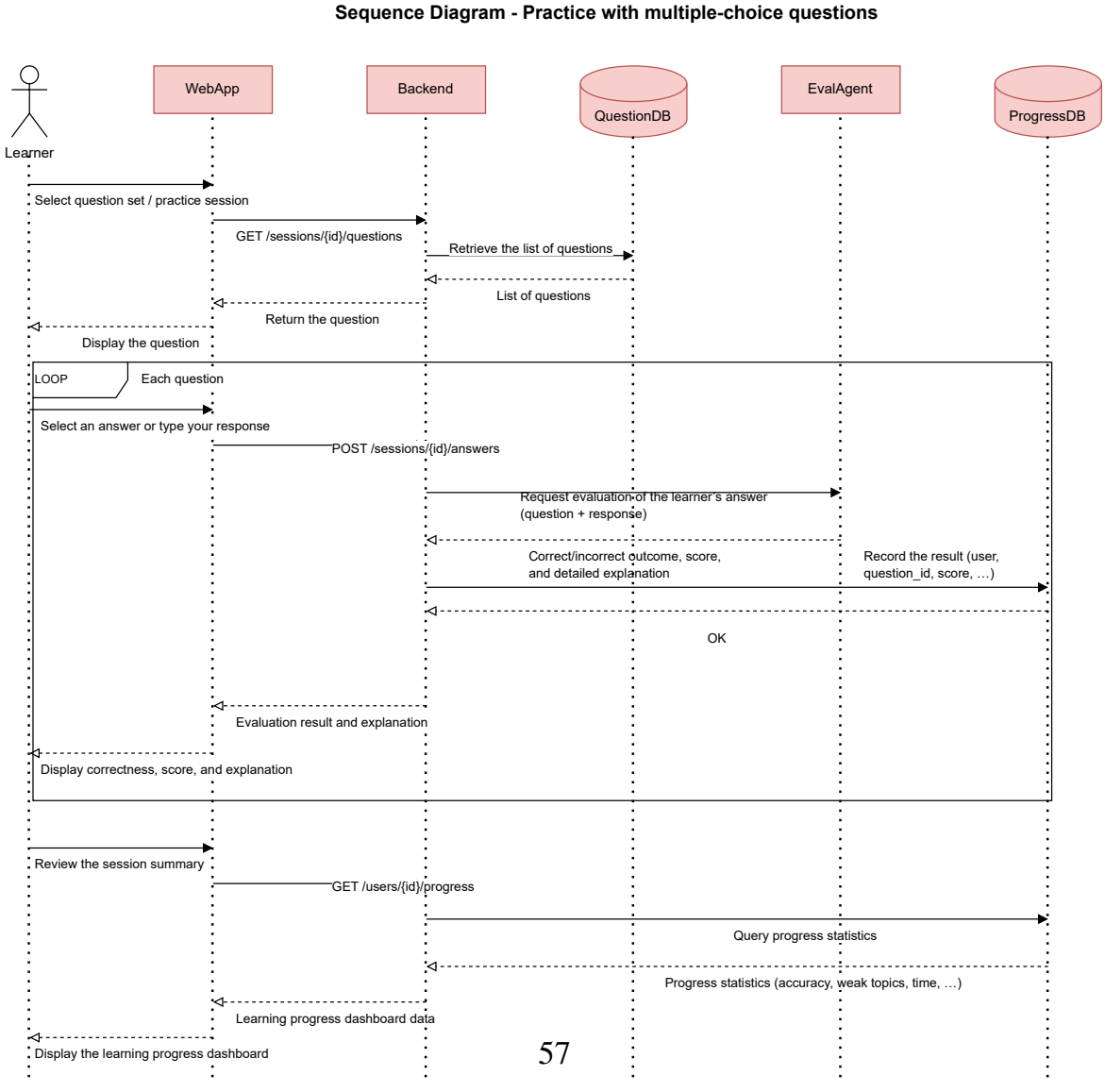


Figure C.1: Initial part of the sequence diagram

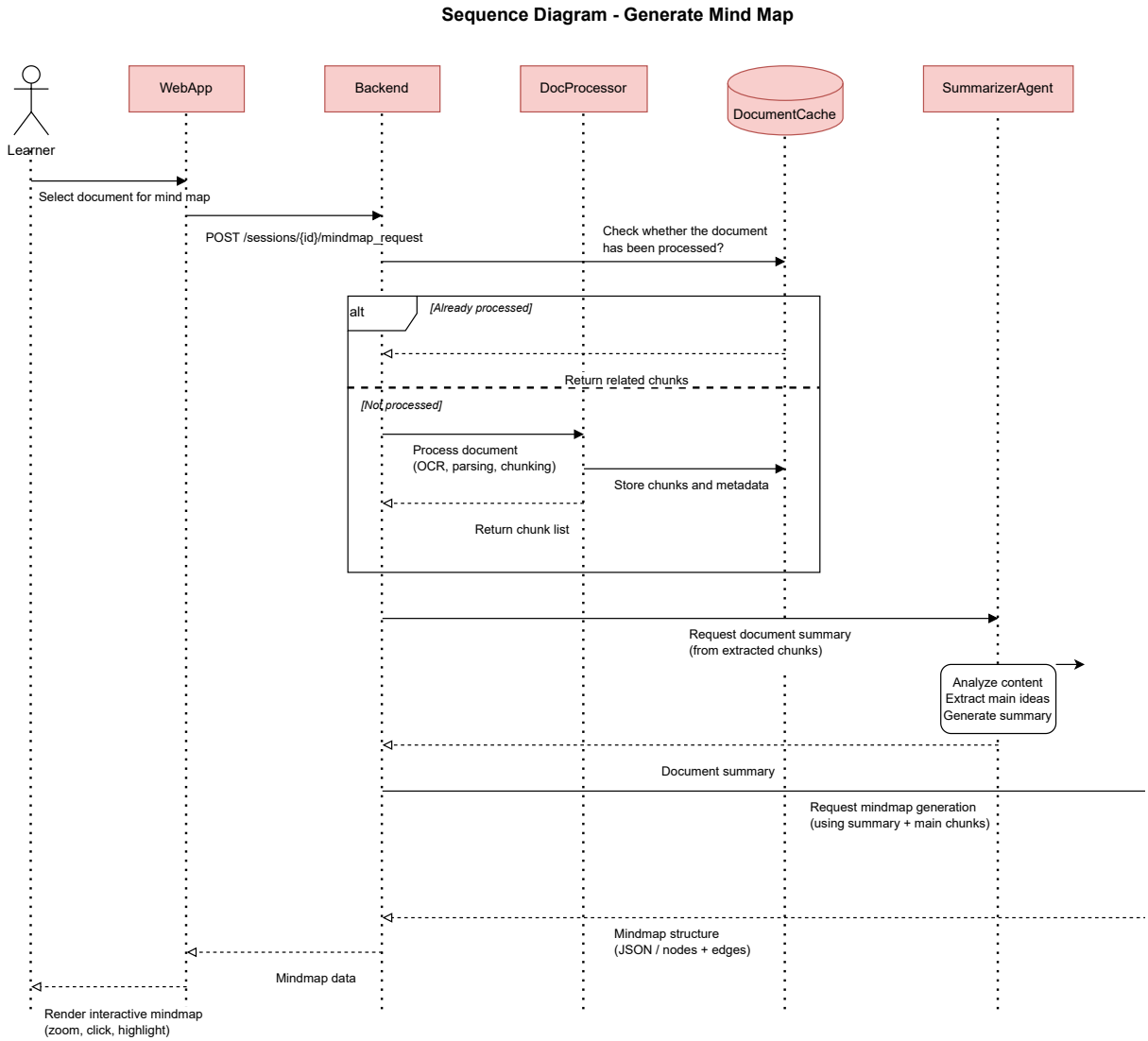


Figure C.2: Final part of the sequence diagram

Appendix D

State Machine Diagrams

Overview

This appendix presents two state machine diagrams that describe the operational lifecycle of the interactive learning system. The diagrams follow the UML State Machine standard and show how the system processes documents, generates questions, supports question–answering, and creates mindmaps. These two diagrams are shown in Figure D.1 and Figure D.2.

1. Learner Behavior State Machine

Figure D.1 describes the interaction cycle of a learner during a working session. The process begins in the *Idle* state, where the user starts the learning module and selects a document to process. After the document is uploaded, the system moves to the *WaitingForProcessing* state. From this point, three main learning functions can be triggered depending on the user's goal:

- **Document Q&A:** The learner asks questions related to the document and receives an answer with supporting context.
- **Question Generation & Practice:** The system automatically generates questions from the document, and the learner practices them and receives feedback.
- **Mindmap Generation:** The document is transformed into a mindmap, which the learner can view, edit, and export.

The diagram also shows error states, return paths to the dashboard, and the full

practice cycle, giving a clear view of how a learner interacts with the system in real usage.

2. Learning Session Lifecycle State Machine

Figure D.2 focuses on the internal processing lifecycle of a learning session. The session begins in the *SessionInitialized* state, then continues through document loading, content processing, metadata creation, and dividing the document into smaller information units (chunks). Once the document is processed and stored in the *ChunkStored* state, the system branches into three independent pipelines:

- **Question Generation Pipeline:** Includes generating questions, reviewing them, refining them, and preparing a question set for practice.
- **Mindmap Pipeline:** Includes creating, viewing, editing, and exporting the mindmap.
- **Document Q&A Pipeline:** Prepares the data structure and serves question–answering requests.

The diagram also includes the states *Failed*, *Completed*, and *Archived*, which represent error handling, successful completion, and session storage.

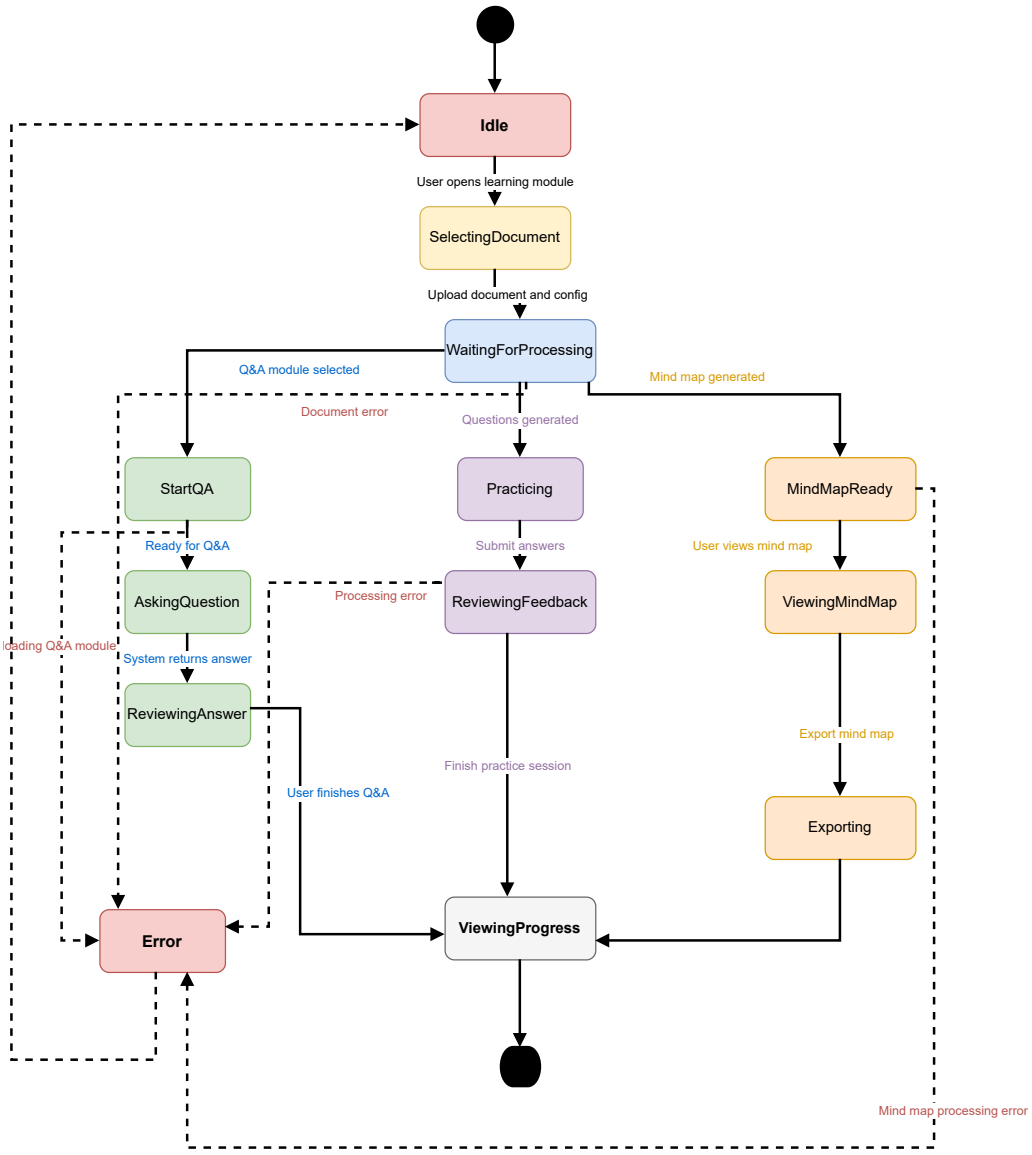


Figure D.1: Learner Behavior State Machine

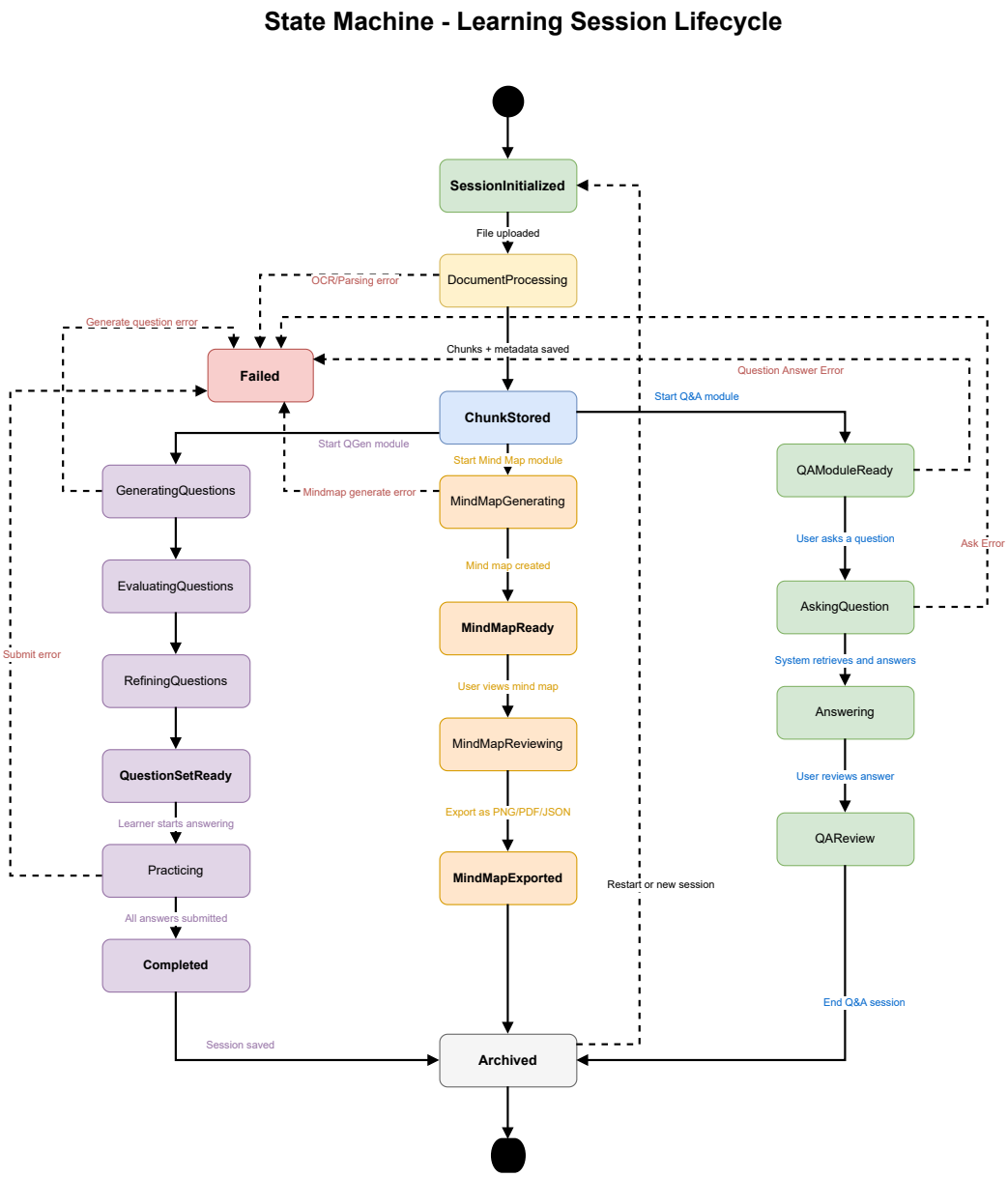


Figure D.2: Learning Session Lifecycle State Machine

Appendix E

Database Design

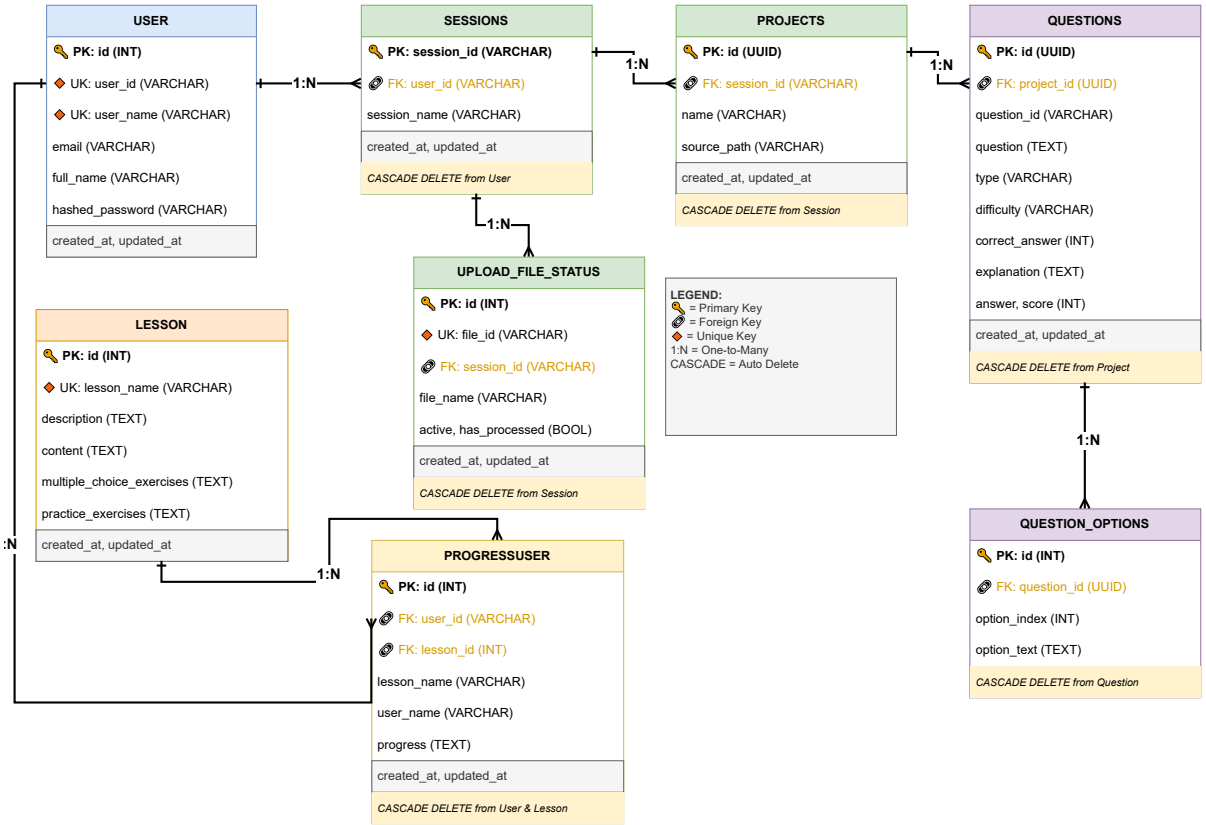


Figure E.1: Database

Appendix F

User Interface Design

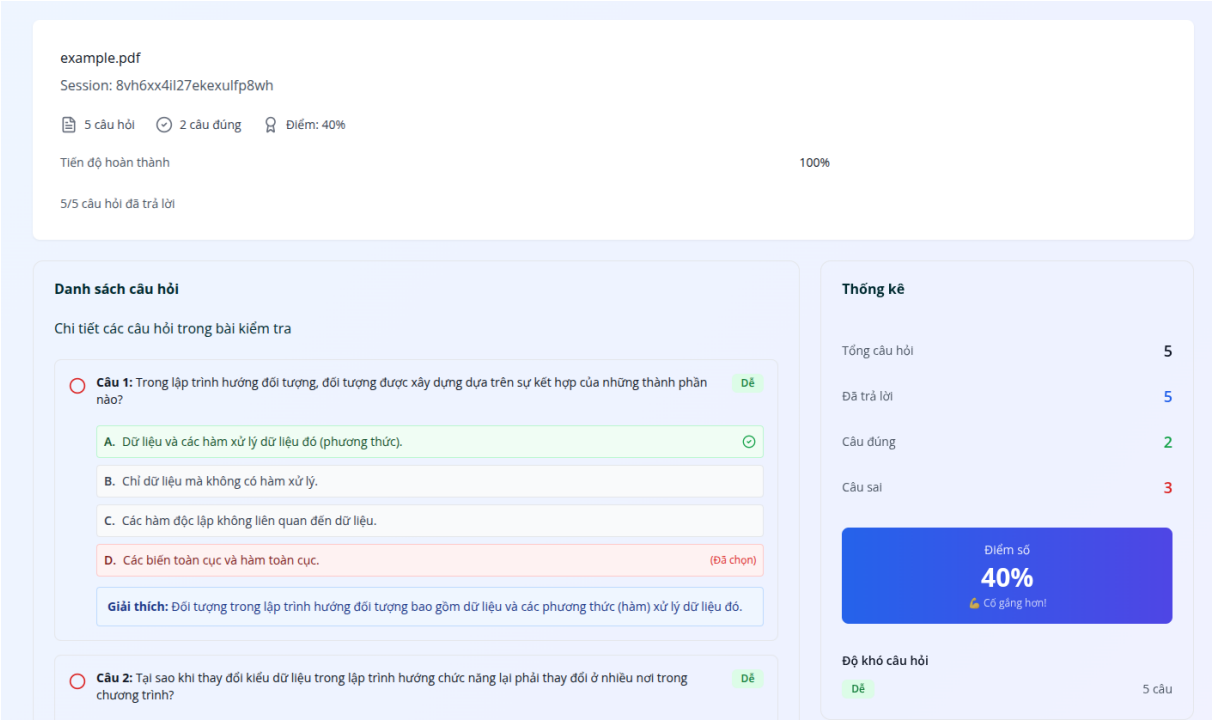


Figure F.1: Illustration of the document mindmap process

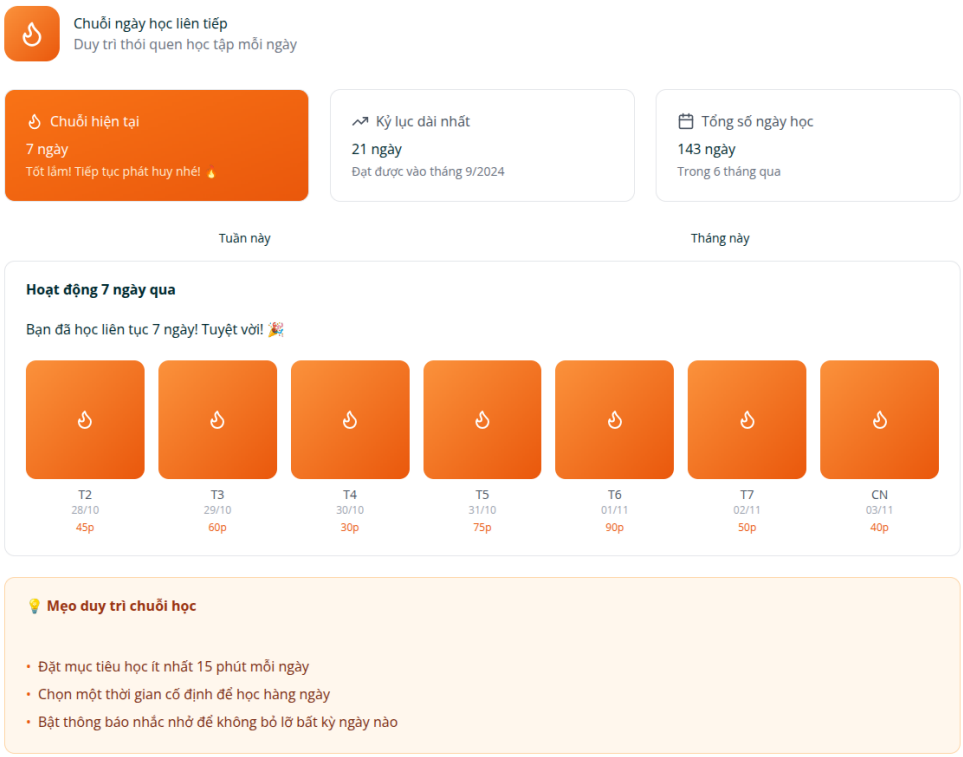


Figure F.2: Illustration of the document mindmap process

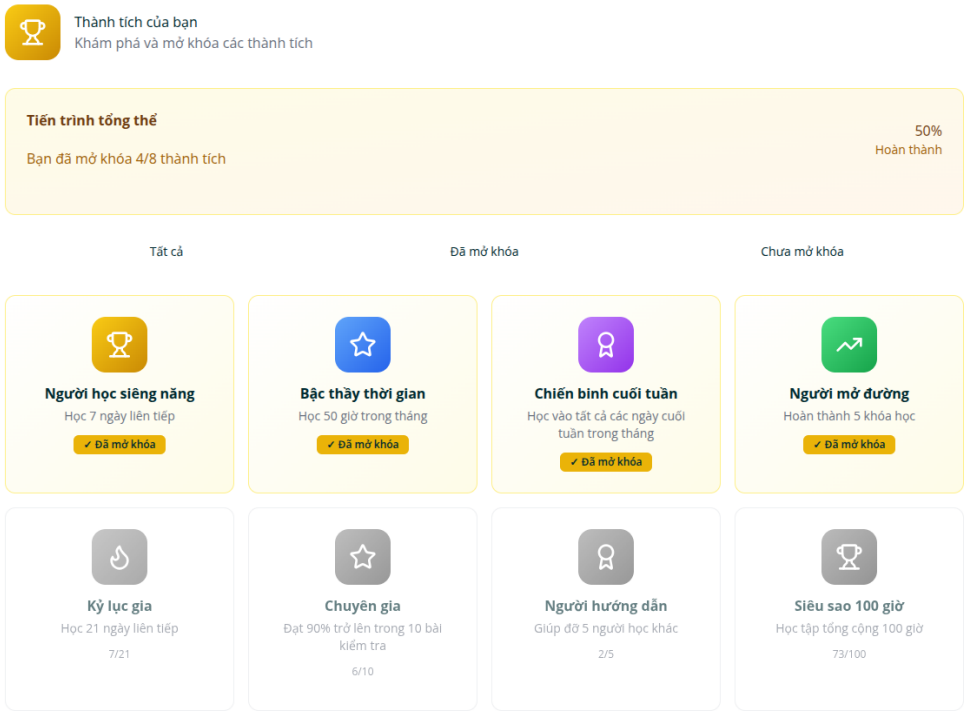


Figure F.3: Illustration of the document mindmap process



Figure F.4: Illustration of the document mindmap process

Appendix G

Test Prompting

Fullset Evaluation Prompt

Nhiệm vụ:
Đánh giá mức độ bản tóm tắt bao phủ các lập luận. Chấm điểm (1 – 4) kèm giải thích ngắn, và trả về JSON.

Hướng dẫn:

- Đọc kỹ các lập luận và bản tóm tắt.
- Đánh giá theo thang điểm ở Bảng 8.
- Xuất JSON gồm:
  - `"explanation"`: Giải thích ngắn.
  - `"rating"`: Điểm số (1–4).

Đầu vào:
Arguments: `{reference_arguments}`
Summary: `{generated_summary}`

Đầu ra:
Chỉ trả về JSON hợp lệ.

Argument Role Evaluation Prompt

Nhiệm vụ:
Xác định bản tóm tắt có hỗ trợ đầy đủ lập luận hay không.

Hướng dẫn:

- Xuất `1`: nếu hỗ trợ đầy đủ lập luận.
- Xuất `0`: nếu thiếu, sai hoặc mâu thuẫn.
- JSON phải gồm:
  - `"decision"`: 1 hoặc 0.
  - `"explanation"`: Giải thích ngắn.

Đầu vào:
Argument: `{argument}`
Summary: `{summary}`

Đầu ra:
Chỉ trả về JSON hợp lệ

Figure G.1: Fullset Evaluation Prompt and Argument Role Evaluation Prompt

Direct Question Generation Prompt

Hướng dẫn:

Dựa trên ngữ cảnh được cung cấp dưới đây, hãy tạo {num_questions} câu hỏi trắc nghiệm nhằm đánh giá mức độ hiểu sâu, khả năng tư duy phản biện và phân tích chi tiết của người học.

Mỗi câu hỏi phải có bốn phương án (A–D) gồm một đáp án đúng và ba nhiễu:

- Phù hợp với ngữ cảnh,
- Liên quan trực tiếp đến nội dung,
- Phản ánh hiểu lầm phổ biến nhưng không mâu thuẫn.

Mỗi câu hỏi chỉ xoay quanh một khái niệm, không gộp nhiều câu.

Ngữ cảnh:

{context}

Understanding Evaluation

Đọc kỹ câu hỏi, các lựa chọn và đáp án đúng. Đánh giá mức độ đo lường hiểu biết của câu hỏi theo thang 1–4: Kiểm tra hiểu sâu; cần tích hợp hoặc vận dụng ý tưởng (4 điểm). Kiểm tra hiểu khái niệm nhưng ở mức trực tiếp, ít tích hợp (3 điểm). Chủ yếu dựa vào ghi nhớ, nhưng có chi tiết yêu cầu suy luận (2 điểm). Kiểm tra ghi nhớ đơn thuần, rất ít yêu cầu hiểu (1 điểm). Chỉ xuất điểm từ 1–4.

Clarity Evaluation

Đọc kỹ câu hỏi, lựa chọn và đáp án. Đánh giá độ rõ ràng theo thang 1–4: Rõ ràng tuyệt đối, không mơ hồ (4 điểm). Hầu hết rõ ràng, có ít mơ hồ (3 điểm). Mơ hồ đáng kể, có thể gây nhầm lẫn (2 điểm). Mơ hồ hoặc khó hiểu (1 điểm). Chỉ xuất điểm từ 1–4.

Figure G.2: Direct Question Generation Prompt and the Evaluation Criteria for Understanding and Clarity

Quality of Choices Evaluation

Đánh giá mức độ đo lường hiểu biết của câu hỏi theo thang 1–4: Kiểm tra hiểu sâu; cần tích hợp hoặc vận dụng ý tưởng (4 điểm). Kiểm tra hiểu khái niệm ở mức trực tiếp, ít tích hợp (3 điểm). Chủ yếu dựa vào ghi nhớ, nhưng có chi tiết yêu cầu suy luận (2 điểm). Kiểm tra ghi nhớ đơn thuần, ít yêu cầu hiểu (1 điểm). Chỉ xuất điểm từ 1–4.

Difficult Evaluation

Đọc kỹ câu hỏi, lựa chọn và đáp án. Đánh giá độ khó theo thang 1–4: Rất khó; yêu cầu hiểu sâu và tư duy cao (4 điểm). Khá khó; cần áp dụng khái niệm (3 điểm). Dễ; chủ yếu dựa vào ghi nhớ hoặc hiểu cơ bản (2 điểm). Rất dễ; trả lời được mà không cần nhiều kiến thức (1 điểm). Chỉ xuất điểm từ 1–4.

Cognitive Level Evaluation

Đánh giá mức độ nhận thức theo thang Bloom, thang 1–4: Yêu cầu tư duy bậc cao (phân tích, tổng hợp, đánh giá) (4 điểm). Yêu cầu áp dụng hoặc hiểu khái niệm (3 điểm). Yêu cầu hiểu cơ bản hoặc nhớ lại (2 điểm). Chỉ kiểm tra ghi nhớ máy móc (1 điểm). Chỉ xuất điểm từ 1–4.

Engagement Evaluation

Đánh giá mức độ hấp dẫn theo thang 1–4: Rất hấp dẫn, kích thích suy nghĩ (4 điểm). Hấp dẫn nhưng không quá đặc biệt (3 điểm). Hơi hấp dẫn nhưng khá đơn giản (2 điểm). Không hấp dẫn hoặc nhàm chán (1 điểm). Chỉ xuất điểm từ 1–4.

Figure G.3: Criteria for Evaluating Multiple-Choice Questions